

hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware

JAN-FREDERIK SCHULTE*, Purdue University, USA
BENJAMIN RAMHORST*, ETH Zurich, Switzerland
CHANG SUN*, California Institute of Technology, USA
JOVAN MITREVSKI, Fermi National Accelerator Lab, USA
NICOLÒ GHIEMMETTI, ENRICO LUPI, DIMITRIOS DANOPOULOS, and VLADIMIR LONČAR†, European Organization for Nuclear Research (CERN), Switzerland
JAVIER DUARTE, University of California San Diego, USA
DAVID BURNETTE, Catapult HLS - Siemens EDA, USA
LAURI LAATU, Imperial College London, United Kingdom
STYLIANOS TZELEPIS, National Technical University of Athens, Greece
KONSTANTINOS AXIOTIS and QUENTIN BERTHET‡, University of Geneva, Switzerland
HAOYAN WANG, PAUL WHITE, and SULEYMAN DEMIRSOY, Altera Corporation, USA
MARCO COLOMBO, Discovery Partners Institute, USA
THEA KLAEBOE AARRESTAD, ETH Zurich, Switzerland
SIONI SUMMERS and MAURIZIO PIERINI, European Organization for Nuclear Research (CERN), Switzerland
GIUSEPPE DI GUGLIELMO, JENNIFER NGADIUBA, JAVIER CAMPOS, BEN HAWKS, ABHIJITH GANDRAKOTA, FARAH FAHIM, and NHAN TRAN, Fermi National Accelerator Lab, USA
GEORGE A. CONSTANTINIDES, ZHIQIANG QUE, WAYNE LUK, and ALEXANDER TAPPER, Imperial College London, United Kingdom
DUC HOANG, NOAH PALADINO, and PHILIP HARRIS, Massachusetts Institute of Technology, USA
BO-CHENG LAI, National Yang Ming Chiao Tung University, Taiwan
MANUEL VALENTIN, RYAN FORELLI, and SEDA OGRENCI, Northwestern University, USA
LINO GERLACH, Princeton University, USA
RIAN BROOKS FLYNN and MIA LIU, Purdue University, USA
DANIEL DIAZ, ELHAM E KHODA, MELISSA QUINNAN, and RUSSELL MARROQUIN SOLARES, University of California San Diego, USA
SANTOSH PARAJULI and MARK S. NEUBAUER, University of Illinois Urbana-Champaign, USA
CHRISTIAN HERWIG, University of Michigan, USA
HO FUNG TSOI and DYLAN RANKIN, University of Pennsylvania, USA
SHIH-CHIEH HSU and SCOTT HAUCK, University of Washington, USA

We present **HLS4ML**, a free and open-source platform that translates machine learning (ML) models from modern deep learning frameworks into high-level synthesis (HLS) code that can be integrated into full designs for field-programmable gate arrays

*Equal contribution.

†Also at Institute of Physics Belgrade.

‡Currently at HEPiA, HES-SO University of Applied Sciences and Arts Western Switzerland.

Authors' Contact Information: Jan-Frederik Schulte, jschulte@cern.ch, Purdue University, USA; Benjamin Ramhorst, bramhorst@ethz.ch, ETH Zurich, Switzerland; Chang Sun, chsun@cern.ch, California Institute of Technology, USA; Jovan Mitrevski, jmitrevs@fnal.gov, Fermi

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

(FPGAs) or application-specific integrated circuits (ASICs). With its flexible and modular design, HLS4ML supports a large number of deep learning frameworks and can target HLS compilers from several vendors, including Vitis HLS, Intel oneAPI and Catapult HLS. Together with a wider eco-system for software-hardware co-design, HLS4ML has enabled the acceleration of ML inference in a wide range of commercial and scientific applications where low latency, resource usage, and power consumption are critical. In this paper, we describe the structure and functionality of the HLS4ML platform. The overarching design considerations for the generated HLS code are discussed, together with selected performance results.

CCS Concepts: • **Hardware** → **Hardware accelerators**; **Reconfigurable logic applications**; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: FPGA, Hardware Acceleration, Machine Learning Acceleration, High-Level Synthesis

1 Introduction

Recent advances in machine learning (ML) and artificial intelligence (AI) have driven the widespread adoption and deployment of neural networks. For example, cloud vendors are increasingly deploying large language models (LLMs) and deep learning recommendation models as part of their services [51, 102]. Similarly, there is an ever-increasing demand for specialized, highly efficient deep learning models for latency- and power-constrained environments, such as real-time systems or edge devices [34]. Some examples include high-energy physics (HEP) systems at CERN [27, 28], robotics [100, 107], network infrastructure [12], industrial manufacturing facilities [101], and satellites and other spacecraft [73, 108]. To tackle the computational and memory requirements of modern neural networks, focus has shifted from conventional CPUs to specialized hardware, such as GPUs, TPUs, and FPGAs [118]. Complementary to specialized hardware, model compression techniques [21, 48] and novel architectures [63, 138] are also explored.

In low-latency, real-time systems, FPGAs and ASICs have become the de facto platforms for accelerating neural network, achieving low latency by implementing deeply pipelined designs, specifically tailored to the target model [83]. Heterogeneously quantized models [30, 113], or extremely low quantization schemes (e.g., binary and

National Accelerator Lab, USA; Nicolò Ghielmetti, nicolo.ghielmetti@cern.ch; Enrico Lupi, enrico.lupi@cern.ch; Dimitrios Danopoulos, dimitrios.danopoulos@cern.ch; Vladimir Loncar, vloncar@cern.ch, European Organization for Nuclear Research (CERN), Switzerland; Javier Duarte, jduarte@ucsd.edu, University of California San Diego, USA; David Burnette, david.burnette@siemens.com, Catapult HLS - Siemens EDA, USA; Lauri Laatu, llaatu@imperial.ac.uk, Imperial College London, United Kingdom; Stylianos Tzelepis, stylianos.tzelepis@cern.ch, National Technical University of Athens, Greece; Konstantinos Axiotis, kaxiotis1.0@gmail.com; Quentin Berthet, quentin.berthet@hesge.ch, University of Geneva, Switzerland; Haoyan Wang, harry.wang@altera.com; Paul White, paul.white@altera.com; Suleyman Demirsoy, suleyman.demirsoy@altera.com, Altera Corporation, USA; Marco Colombo, mcolom4@uillinois.edu, Discovery Partners Institute, USA; Thea Klæboe Aarrestad, thea.aarrestad@cern.ch, ETH Zurich, Switzerland; Sioni Summers, sioni@cern.ch; Maurizio Pierini, maurizio.pierini@cern.ch, European Organization for Nuclear Research (CERN), Switzerland; Giuseppe Di Guglielmo, gdg@fnal.gov; Jennifer Ngadiuba, ngadiuba@fnal.gov; Javier Campos, jcampos@fnal.gov; Ben Hawks, bhawks@fnal.gov; Abhijith Gandrakota, abhijith@fnal.gov; Farah Fahim, farah@fnal.gov; Nhan Tran, ntran@fnal.gov, Fermi National Accelerator Lab, USA; George A. Constantinides, g.constantinides@imperial.ac.uk; Zhiqiang Que, z.que@imperial.ac.uk; Wayne Luk, w.luk@imperial.ac.uk; Alexander Tapper, a.tapper@imperial.ac.uk, Imperial College London, United Kingdom; Duc Hoang, dhoang@mit.edu; Noah Paladino, npaladin@mit.edu; Philip Harris, pharris@mit.edu, Massachusetts Institute of Technology, USA; Bo-Cheng Lai, bclai@nycu.edu.tw, National Yang Ming Chiao Tung University, Taiwan; Manuel Valentin, manuel.valentin@u.northwestern.edu; Ryan Forelli, rforelli@u.northwestern.edu; Seda Ogrenci, seda@northwestern.edu, Northwestern University, USA; Lino Gerlach, lino.oscar.gerlach@cern.ch, Princeton University, USA; Rian Brooks Flynn, rbflynn@purdue.edu; Mia Liu, liu3173@purdue.edu, Purdue University, USA; Daniel Diaz, d4diaz@ucsd.edu; Elham E Khoda, ekhoda@ucsd.edu; Melissa Quinnan, mquinnan@ucsd.edu; Russell Marroquin Solares, rmarroquinsolares@ucsd.edu, University of California San Diego, USA; Santosh Parajuli, santoshp@illinois.edu; Mark S. Neubauer, msn@illinois.edu, University of Illinois Urbana-Champaign, USA; Christian Herwig, herwig@umich.edu, University of Michigan, USA; Ho Fung Tsoi, hftsoi@sas.upenn.edu; Dylan Rankin, dsrankin@sas.upenn.edu, University of Pennsylvania, USA; Shih-Chieh Hsu, schsu@uw.edu; Scott Hauck, hauck@uw.edu, University of Washington, USA.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1936-7414/2026/4-ART

<https://doi.org/10.1145/3801979>

ternary models [85, 132]), take advantage of their high configurability to map operations directly to custom logic elements. FPGAs, in particular, have been deployed in various real-time systems [7, 8, 24, 27, 28], due to their high flexibility and lower entry barrier compared to ASICs. However, traditional FPGA development workflows, even when using higher levels of abstractions, such as High-Level Synthesis (HLS), are poorly suited to adapt the functionality of modern deep learning frameworks (e.g., PyTorch [11], TensorFlow [3]) as they mainly target CPUs and GPUs. Therefore, deploying neural networks on FPGAs still requires considerable experience and significant time investment.

Aiming to solve these issues, there has been a growing number of platforms automating neural network deployment on FPGAs (see Section 12). While these platforms raise the level of abstraction and represent a significant step forward, each platform provides a range of somewhat arbitrary and application-specific features. For example, many of these platforms only support models defined in a single deep learning framework, or target only a single type of FPGA [9, 42, 66]. Additionally, the resulting hardware implementations are typically tailored to specific model architectures and precisions, such as convolutional neural networks (CNNs) [55, 110, 125, 127, 128, 131] or multilayer perceptron (MLPs) [103, 115]. Moreover, these platforms rarely allow the user to choose between different implementations of the models on hardware, which significantly reduces flexibility when balancing latency and resource consumption. Finally, many of these frameworks [53, 103, 115] are no longer actively maintained.

In this work, we present HLS4ML [45]¹, a free and open-source, easy-to-use, and modular platform that translates models from common deep learning frameworks (e.g. PyTorch, Keras), into low-latency, dataflow designs for FPGAs and ASICs. HLS4ML acts as a compiler, providing user-facing *front ends* that parse trained models and translate them into an *internal representation* (IR), which is iteratively optimized before hardware lowering through a set of *optimizers*. *Back ends* targeting different HLS compilers map the optimized IR to HLS implementations of layers and operators, and create IP cores for the target model that can be synthesized and integrated into larger applications.

HLS4ML supports all major deep learning frameworks, including PyTorch [11], Keras [23], and ONNX [13], as well as their quantized counterparts: QKeras [30], HGQ [113], Brevitas [87], and QONNX [88, 122]. On the back end side, HLS4ML supports a growing set of HLS compilers from different vendors, including Vitis HLS [6] from AMD and OneAPI [65] from Intel. In addition to designs for FPGAs, HLS4ML facilitates the deployment of neural networks on ASICs [38, 89] through Catapult HLS [106]. Model architectures that have been implemented using HLS4ML include MLPs [40], CNNs [2, 47], recurrent neural networks (RNNs) [74], graph neural networks (GNNs) [43, 64, 86], and transformers [69, 70]. Additionally, the Extension API allows users to easily add support for any missing layers or operators, while still leveraging the rest of HLS4ML's implementations and optimizations. Configuring the implementation strategy, hardware parallelization, and variable precision, users can optimize their designs for latency or resource usage without modifying or understanding the HLS code directly. This flexibility allows for rapid prototyping and the co-design of models and hardware. Finally, HLS4ML provides multiple environments for model verification. In addition to conventional software emulation, HLS4ML enables users to directly deploy their model on specific FPGAs (e.g., Zynq or Alveo cards) with a few lines of Python code. Owing to HLS4ML's modularity and standardized interfaces, it is straightforward to integrate it with any platform for FPGA deployment, with out-of-the-box support for commercial [5, 65] or academic [97] shells.

As a platform, HLS4ML can be used in two ways: first, as a deployment platform for various applications, including HEP experiments [27, 28], quantum computing [14, 37], network firewalls [60], self-driving cars [47], heart signal monitoring [80], and space exploration [108]. Second, HLS4ML can be used as a starting point for research in efficient deep learning and hardware-software co-design [92–94, 98, 113, 119, 120, 135]. Originally

¹Source code and documentation available at <https://github.com/fastmachinelearning/hls4ml/> and <https://fastmachinelearning.org/hls4ml/index.html>.

presented in [40] for HEP applications at CERN, HLS4ML has since grown into a widely used, open-source project for deep learning acceleration and research on custom hardware (see Section 10 for more details on HLS4ML use cases and applications). The project is actively maintained with contributions from both academia and industry, is well documented and tested, and frequent supporting tutorials, workshops, and seminars are held to foster the user community.

2 Background

FPGAs have emerged as suitable platforms for low-latency, low-power neural network inference, due to their low-level hardware control and high configurability. While development for ASICs is also supported in HLS4ML, FPGAs are the primary target device and the focus of this paper. In the following, we present a brief overview of common design techniques for model inference on FPGAs. More in-depth overviews are given elsewhere [18, 62, 83].

To achieve high throughput and low latency inference, there are many established design techniques. *Parallelization* splits the calculations between multiple processing units (PUs), for example by partitioning the input data between different instances of the layer implementations (data parallelization) or distributing the computation for the neurons of a layer between PUs (model parallelization). *Pipelining* partitions the calculation in depth so that, for example, subsequent layers are assigned to different PUs, allowing continuous data flow through the design. This allows the design to accept new inputs before the overall inference is completed, resulting in a low initiation interval (II). Most frameworks [15, 40, 110, 125] use both techniques to achieve maximum performance.

Many frameworks [40, 55, 123] store weights and intermediate results in on-chip memory to avoid the overhead of accessing off-chip memory. These implementations can achieve very low latency and high throughput for models that fit within the limited on-chip logic resources. For larger models, such as transformers, previous studies [54, 68, 124] have proposed using high-bandwidth memory (HBM) on recent FPGAs.

For computations, many FPGAs include hardened digital signal processors (DSPs) optimized for multiply-accumulate (MAC) operations with wide bit-widths in deep learning models. Compared to the higher abstraction in CPU and GPU programming, arithmetic operations with these blocks reduce instruction overhead and enable more granular control over data flow. Look-up tables (LUTs) and, sometimes, fast carry chains allow implementing MAC operations without the need for dedicated multiplier blocks like DSPs, either by shift-and-add operations, such as the Booth multiplication algorithm [16], or by table look-ups for small bit-width multiplications. Recent works [10, 121] propose training neural networks that can be mapped directly to FPGA LUTs, often achieving high operating frequencies and low resource usage with minimal accuracy loss.

Compared to the commonly used floating-point precisions on CPUs and GPUs, FPGA designs typically represent variables in fixed-precision with lower bit-width. Quantizing a model trained with full precision to lower bit-widths with no or little retraining is known as post-training quantization (PTQ). Higher degrees of quantization can be achieved while maintaining accuracy by training the model directly with the intended bit-width, a technique known as quantization-aware training (QAT). Quantization is a method particularly suited for FPGA acceleration, as the arbitrary-precision operation can be efficiently mapped to low-level logic elements in hardware. Examples of quantization for FPGAs include heterogeneously quantized models [30, 113], as well as binary and ternary models [85, 121, 132]. Additionally, models can also be compressed by pruning parameters, setting some weights or activations to zero. The level of pruning is selected to balance model size with an acceptable inference accuracy loss [21, 92]. On FPGAs, pruning can be structured to be compatible with the low-level hardware implementation [98].

Traditionally, register-transfer level (RTL) design in languages such as VHDL, (System)Verilog, or Chisel have been used to program FPGAs. While allowing for more granular control, producing designs in this way is usually more challenging and comes with a steep learning curve. *High-level synthesis* (HLS) abstracts FPGA programming

significantly by compiling high-level C/C++ or SystemC code into RTL code for specific target hardware using an *HLS compiler*, such as Vitis HLS or CATAPULT HLS. Preprocessor directives, or so-called *pragmas*, are used to guide the compiler in the hardware implementation of the design. This significantly reduces the difficulty of implementing complex designs for ML inference. While this approach sacrifices some direct control over exact implementations, HLS has been shown to achieve performance on par with hand-written RTL designs [71, 72] for neural networks (Section 9.6).

3 Overview

To achieve high performance, high portability, and easy extensions in the fast-moving field of ML, HLS4ML is structured in a modular fashion, mimicking modern compilers. HLS4ML includes a set of *front ends*, which parse high-level models into an internal representation (IR), a set of *optimizer passes*, which optimize the model graph for the target hardware, and a set of *back ends*, which create high-performance IPs for different flavors of HLS targeting FPGAs from different vendors. Additionally, it includes a rich set of features for model simulation and hardware validation on FPGAs/SoCs.

The flow from the initial model to the FPGA IP is illustrated in Figure 1. The users provide a trained model and a configuration file that specifies options such as parallelization factors, variable precisions, and target clock period, which are then parsed by the front-end parsers.² Generally, HLS4ML supports fixed-point, exponential, ternary, and binary data types. In the case of PTQ, variable quantization is determined by the precision set in the user configuration, and in the case of QAT, it is automatically inferred from the quantized model (e.g., from QKERAS, QONNX). Following that, the model graph is iteratively optimized and refined, given the user configuration and the target back end. Finally, an FPGA/HLS project containing the generated HLS code and the supporting scripts for simulation, synthesis, and deployment is returned to the user.

HLS4ML supports all major deep learning frameworks through dedicated front ends for KERAS, PyTorch, and ONNX. Additionally, HLS4ML is able to process quantized models from these frameworks; namely models implemented in QKERAS, HGQ, BREVITAS, and QONNX. We describe the front ends in more detail in Section 4. On the back-end side, HLS4ML keeps the same design principles of modularity and extendability. Currently available are the *Vitis* and *Vivado* back ends for Vitis HLS, and its deprecated counterpart VIVADO HLS, for AMD/Xilinx FPGAs, the *oneAPI* and *Quartus* back ends for oneAPI, and its deprecated counterpart INTEL HLS, for Intel/Altera FPGAs, and the *Catapult* back end for CATAPULT HLS, targeting the Siemens EDA flow. These back ends are discussed in more detail in Section 6. To translate between the different front and back ends, HLS4ML converts the model into an IR. The IR initially replicates the structure of the model before a set of optimizers modifies it to better suit the hardware architecture and performance requirements. The IR and optimizers are described in more detail in Section 5. Following a similar modular approach, HLS4ML implements a library of HLS kernel templates, each implementing the functionality of a model layer or parts thereof. Here, each back end has its own set of templates, which is hand-optimized for the target HLS compiler. To allow greater flexibility and design space exploration different templates are often available that minimize either latency or resources. The user has full control over which of these configurations is used and can also control the degree of parallelism and variable precisions, both of which can be set on a per-layer basis.

Finally, it is important to note that public releases of HLS4ML support most types of commonly used neural networks. MLPs and CNNs are fully supported in all front and back ends. RNNs are also supported, though currently not in the ONNX front end. While some graph neural networks (GNNs) have been realized using HLS4ML [64, 86, 95], support for generic GNNs is currently still in development. Specifically, no support for any PyTorch GEOMETRIC functionality is currently available. Support for multi-head attention (MHA) and Linformer

²For more information on the configuration parameters and commands to run these steps, see <https://fastmachinelearning.org/hls4ml/api/configuration.html> and <https://fastmachinelearning.org/hls4ml/intro/setup.html>.

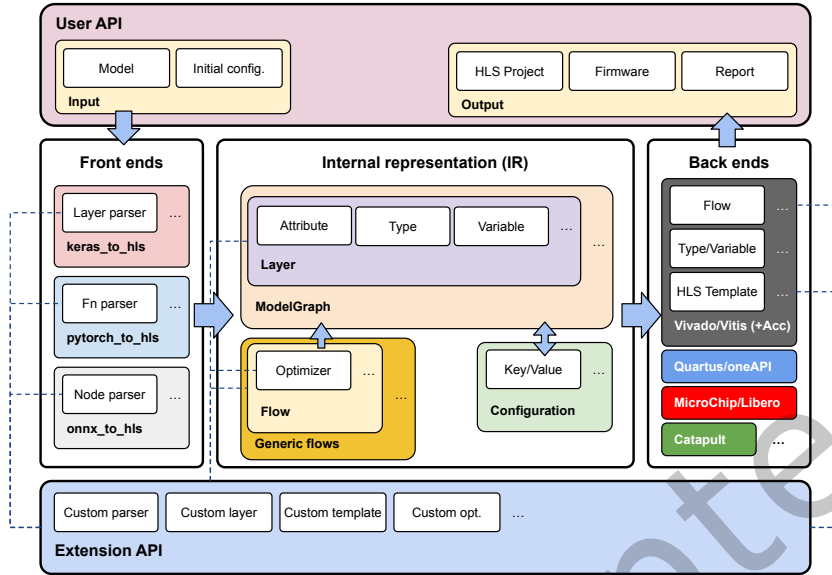


Fig. 1. Model conversion and compilation flow in HLS4ML.

transformer models has been introduced in HLS4ML v1.2.0 via the HGQ 2 front end and is currently only available in the Vitis back end [76]. A detailed overview of the availability of common operators is given in Tables 1 and 2. In addition, HLS4ML supports a large number of activation functions, such as ReLU or softmax, as well as general tensor operations (e.g. reshaping, concatenation, or addition). Even though many layers are supported, a model may not necessarily be suitable for acceleration with HLS4ML. Very large or complex models can fail to pass HLS compilation due to the on-chip, low-latency dataflow implementation of HLS4ML designs which can lead to high resource utilization and congestion (more details on the limitation of HLS4ML can be found in Section 11). Such models should be compressed through quantization and pruning, or, in some cases, consider alternative acceleration techniques, leveraging for e.g., overlay architectures or off-chip memory. If a model contains currently unavailable operations/layers, HLS4ML offers an Extension API to add support for additional layers (Section 8), while still relying on the implementations for existing layers, optimizers, and scripts.

4 Front ends for different ML libraries

The front end converts models from the supported frameworks mentioned above into the IR. For each deep learning framework, there is a dedicated front end that (1) parses the model graph, and (2) extracts relevant information from each individual layer or operation. The second step is implemented with an extensive repository of layer handlers, one for an individual, or a family of supported layers or operations. Each layer handler accepts a layer object and returns a dictionary with the necessary configuration and data for representing the layer in HLS4ML's IR. All weights in the model are converted to NUMPY arrays at this stage, and front-end specific objects are eliminated for compatibility.

4.1 Keras

Traditionally, HLS4ML has focused on models created in KERAS 2 and supports the most commonly used layers in this framework. For KERAS 2, both Keras objects and serialized representations of the model, which contain detailed

Table 1. Support for different NN layers or types in the HLS4ML front ends. Support is indicated by ☒, while not supported layers are marked ☐. Layers not supported in the respective NN frameworks are marked –.

Layer	Front end							
	KERAS 2	QKERAS	HGQ	KERAS 3	HGQ 2	PyTORCH	ONNX	QONNX
Linear/Dense	☒	☒	☒	☒	☒	☒	☒	☒
1D/2D Convolution	☒	☒	☒	☒	☒	☒	☒	☒
LSTM/GRU	☒	☒	–	☒	–	☒	☐	☐
Einsum	☐	–	–	☒	☒	☒	☐	–
Multihead Attention	☐	–	–	☐	☒	☐	☐	–
Batch Normalization	☒	☒	☒	☒	☒	☒	☒	☒
Layer Normalization	☒	☐	☐	☐	☐	☒	☐	☐
GNN	☐	☐	☐	☐	☐	☐	☐	☐

Table 2. Support for different layers in the HLS4ML back ends. Support is indicated by ☒, while not supported layers are marked ☐.

Layer	Back end		
	Vitis/Vivado	Quartus/oneAPI	Catapult
Linear/Dense	☒	☒	☒
1D/2D Convolution	☒	☒	☒
LSTM/GRU	☒	☒	☒
Einsum	☒	☐	☐
Multihead Attention	☒	☐	☐
Batch Normalization	☒	☒	☒
Layer Normalization	☒	☐	☐
GNN	☐	☐	☐

information about the model architecture and layer configurations, are accepted. This allows the conversion of models that are saved in the HDF5 format without requiring the Keras library to be installed. Direct parsing of .keras files is not supported due to the absence of a public API for reading model configurations. However, such models can be loaded in Keras and passed to HLS4ML as in-memory objects. KERAS 3 introduces major changes to the KERAS library. Therefore, the KERAS 3 parser does not inherit from the KERAS 2 parser. The KERAS 3 parser directly ingests a `keras.api.Model` object, and reconstructs the model architecture via data dependency solving. KERAS 3 layer handlers also ingest `keras.api.layers.Layer` objects directly, and return a dictionary with the layer configuration and weights. When a layer type is not supported, the parser will attempt to fall back to the KERAS 2 layer handler (if available) to extract the configuration.

The KERAS 2 parser supports models quantized by two different QAT libraries, QKERAS and HGQ, while the KERAS 3 parser currently only supports HGQ. For quantized layer types, the quantization parameters are included in the IR of the quantized layer configuration. Quantization parameters derived from QAT libraries are enforced in HLS4ML and will override any user-provided configurations. QKERAS supports three major quantization types: fixed-point integer based, binary, and power-of-two quantization, of which are all supported by HLS4ML, while the power-of-two quantization may only be used for the weights. For HGQ models, a dedicated optimizer pass in the IR is invoked for precision configuration. This involves symbolic precision propagation through the model, and all quantization parameters are derived from explicit quantization layers converted from HGQ, along with the weights provided. User-provided precision configurations are discarded when converting from HGQ, and

conversions from properly defined HGQ models are always bit-exact up to the floating point representation limits.

4.2 PyTorch

Symbolic tracing in the TORCH FX framework is used to infer the architecture and configuration of PyTorch models. In general, HLS4ML supports operations implemented as both `torch.nn.Module` objects and stateless operators in the `torch.nn.functional` module. The support for PyTorch layers generally matches that of KERAS 2. A custom tracer inheriting from the `TORCH.FX.TRACER` class is used to enable the parsing of custom layers in the Extension API (Section 8). Since HLS4ML was first developed to support KERAS 2 models, which adopts the “channels-last” convention for multi-dimensional tensors, many HLS kernel templates only have a “channels-last” version implemented. PyTorch uses the “channels-first” convention in most of its operations. Therefore, an optimizer adds the necessary transpose operations to the HLS4ML IR to ensure compatibility with the “channels-last” convention.

Models trained using QAT in BREVITAS can be converted to the QONNX format using the BREVITAS-ONNX export functionality, allowing them to be ingested by the ONNX front end described below. This approach is also common in other frameworks [15, 123]. In addition, support for direct parsing of BREVITAS models, without reliance on QONNX, as part of the PyTorch front end is in development.

4.3 ONNX

ONNX models are supported with direct object parsing. The converter ingests an ONNX object and parses its graph to determine the model inputs, outputs, and all operators present in the model. Because of the fine-grained representation of operators in ONNX models, HLS4ML requires the model to be preprocessed, usually referred to as “cleaning”, before conversion. Cleaning is performed using the QONNX [122] package, which performs constant folding, shape inference, renaming of the nodes into human-readable names, and conversion of the graph into the “channels-last” convention. Additionally, QONNX, as an extension of the ONNX library, introduces three new operators: `QUANT`, `BIPOLARQUANT`, and `TRUNC` to enable flexible representation of quantized models. HLS4ML supports a subset of models representable by QONNX that are physically feasible for FPGA implementation. In the HLS4ML IR, the precision is derived from the quantization operators and enforced. Additional scaling or unscaling is applied as needed. Figure 2 illustrates a QONNX representation of a simple one-layer MLP with a softmax output layer, compared to the representation in QKERAS.

5 Internal representation and optimizer flows

5.1 Internal Representation

The HLS4ML IR aims to provide a front- and back-end agnostic representation of models. By abstracting models from different frameworks into a uniform representation, HLS4ML can apply transformations and optimizations to a model independent of the framework used to create it. The IR is implemented as a `ModelGraph` object where each node corresponds to a layer or operator in the network. Each node contains all layer-specific information such as operation type, weights, quantization method, and connections to other nodes. During conversion, the IR is initialized with the layers from the original model by the front-end converter. Then, the IR undergoes a series of transformations performed by the *optimizer flows* to reach the final state used in the code generation of the target back end.

By default, HLS4ML creates a single HLS project for the entire model, which is then synthesized into a monolithic design. As of v1.2.0, HLS4ML supports a new feature that allows for the splitting of the model into multiple subgraphs. This is implemented by extending the `ModelGraph` with the `MultiModelGraph` object, which splits the model at user-defined layers, and creates independent HLS projects that can be synthesized independently.

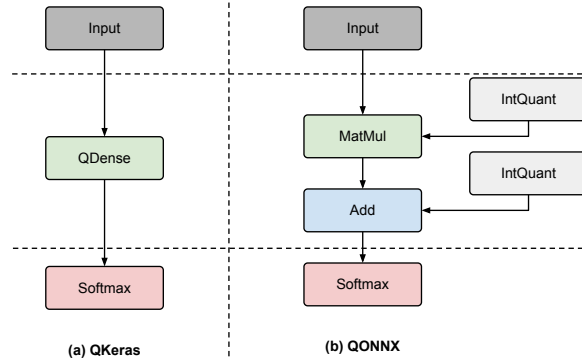


Fig. 2. Comparison of the representation of an example model consisting of a one-layer MLP with a softmax output layer in QKERAS (left) and QONNX (right).

This allows for parallel synthesis to reduce synthesis time, or potentially distributing large models over multiple chips or devices in the future. HLS4ML currently supports automatic stitching of the synthesized IP cores from subgraphs into a single design for the Vivado/Vitis back ends. By running the individual synthesis in parallel, the HLS synthesis time for a quantized ResNet for CIFAR-10 classification decreases from 7 h to 3 h on average.

5.2 Optimizer flows

Inspired by modern compilers, HLS4ML implements a series of transformations, called *optimizer flows*, that iteratively transform the IR for the target back ends, where each flow represents a distinct optimization stage. Optimizer flows can either be generally applicable to all models or specific to a target back end. A single flow implements multiple *optimizers*, each performing a specific transformation on the model, such as precision propagation, template instantiation, or removing redundant operations. For instance, an optimizer may fuse batch normalizations that immediately follow an affine projection (fully connected or convolutional layers) operation into a single node with equivalent, fused weights when both nodes are not quantized.

5.3 User directives

In addition to the graph representation of the model itself, the `ModelGraph` object also holds directives regarding the model conversion and optimization process in the form of an `HLSSConfig` object. The `HLSSConfig` object contains information that cannot be derived directly from the model, such as the target back end, implementation strategy, I/O type, and, in some cases, quantization precision and resource reuse preferences. Users provide these configurations as a Python dictionary. Some of the hardware-oriented configurations (e.g., I/O type, strategy) are further discussed in Section 6.1, in conjunction with the hardware implementation of neural network kernels.

Quantization parameters are a crucial part of the model conversion process. Improperly configured quantization parameters may introduce extra errors that accumulate throughout the model, which can lead to significant performance degradation. Currently, HLS4ML supports fixed-point, exponential (power-of-two), ternary, and binary data types. When the front-end framework quantizes the model properly, the quantization parameters will be stored directly in the nodes. However, some precision information may not always be available depending on the front end used. For instance, the accumulator³ precision is rarely available from the front-end libraries. Since v1.0.0, HLS4ML is able to automatically determine it through conservative estimation to avoid unwanted

³The accumulator holds the intermediate results in a layer for the MAC operations.

overflows when the user specifies "auto" for the accumulator precision in the configuration. Other precision types, such as the weight, bias, and result precision, can also be inferred automatically, depending on the layer type and the metadata it contains when "auto" is specified. Since v1.2.0, HLS4ML includes a dedicated optimizer pass that propagates precision at the model level for bit-exactness when the model is fully quantized, which relies only on explicitly defined quantizers and the weights provided through interval arithmetic. User-supplied precision configurations are ignored when this optimizer pass is enabled.

6 Back ends and hardware implementations

HLS4ML generates HLS code through a set of manually optimized templates for different layers and operations, which are combined to represent a complete model to be synthesized. Each back end includes a set of dedicated templates and optimizers. Some back ends, such as those for VIVADO HLS and VITIS HLS, may share some templates and optimizers because of their similarity. The optimizers will register the back-end specific templates and apply them to the IR. They also perform back-end specific optimizations, such as pragma insertion, precision propagation, constant folding, and others. When creating an HLS project, the back ends will also generate the necessary utility files such as TCL scripts for synthesizing the project, test benches, and bridge code for calling the generated libraries directly from high-level Python functions. After synthesizing the generated HLS code, the back ends can parse the synthesis reports and display a concise summary of the resource usage and latency.

6.1 Hardware implementations

In the following, we provide a brief overview of hardware implementations in HLS4ML, with an emphasis on constant matrix-vector multipliers (CMVM) and activations, which are core building blocks of neural networks. Further details on specific layer implementations can be found in the corresponding publications for MLPs [40], CNNs [2, 47] and RNNs [1, 74]. In addition, we discuss the various strategies and data flow options supported in HLS4ML, each of which provides a trade-off between the resources used and inference latency.

Constant matrix-vector multiplication, strategies, and parallelism: The CMVM operation is a fundamental building block of neural networks. HLS4ML places significant emphasis on its efficient implementations, with three supported strategies: *Latency*, *Resource*, and *Distributed Arithmetic*. To control the degree of parallelism, HLS4ML allows users to set the *Reuse Factor* (RF). Given a weight matrix of size $M \times N$ and a reuse factor RF , HLS4ML implementations of CMVM are such that there are no more than $N_MULT = \frac{M \cdot N}{RF}$ multiplications per clock cycle (e.g., Fig. 3a). Different strategies will differently impact latency and resource consumption of a layer, and users are advised to choose based on their design goals. The availability of a particular strategy for a layer varies across back ends.

The *Latency* strategy implements fully unrolled CMVM with weights directly embedded into multiplier circuits, allowing the HLS compiler to perform intensive optimizations, such as removing multiplications with zeros, replacing multiplications with specific constants with shift-and-add operations, or reordering the multiplications to improve timing. The RF is implemented as a compiler directive that limits the number of parallel multiplications and determines the initiation interval of the CMVM operation. The HLS compiler determines the exact ordering and hardware primitive for the MAC operations, based on the data type, precision, and other conditions (e.g. target hardware and frequency). In cases where DSP blocks or other hardened multipliers are used for the MAC operations, a larger RF will usually result in reusing those blocks and lower resource consumption. However, when MAC operations are implemented in fabric, such as LUTs and fast carriers with shift-and-addition operations, a higher RF will not necessarily result in resource reduction, but will still increase the II. Depending on the underlying compiler and hardware mappings, the resource usage may even increase. Therefore, the *Latency* strategy is suitable for small models, targeting very low inference latencies. Additionally, the RF should be tuned carefully as it may not yield resource savings and may increase II.

In the *Resource* strategy, the CMVM implementation stores the weights in BRAM and explicitly controls the degree of parallelism in by specifying the number of MAC operations performed in parallel per clock cycle (Fig. 3a). Weights stored in BRAM are partitioned into *RF* blocks, each of size N_MULT (Fig. 3b). Each clock cycle, a slice of N_MULT weights is accessed and processed together with the inputs by a MAC unit, leading to an II of *RF* clock cycles. Depending on the chunk size N_MULT , the weights may be stored in multiple BRAMs to increase the number of ports for parallel read. The exact implementation of the MAC unit will depend on the bit-width of the weights and inputs, as well as compiler heuristics. In the case of DSP-based MAC units, it can be implemented as cascaded DSP tiles, whereas for LUT-based MAC units, the exact implementation differs heavily depending on the precisions, target frequency and other factors. However, for specific operand types, HLS4ML can overwrite compiler primitives; for e.g. multiplications with power-of-two weights can be implemented using shifters explicitly, while multiplications with binary and ternary weights can be realized with AND and XOR gates. Increasing the *RF* in the Resource strategy directly reduces the number of MAC units used, thus reducing resource consumption. The Resource strategy is better suited for large models, since the Latency strategy can fail to compile in these cases due to full loop unrolling. However, due to the overhead in data movement and the use of general MAC units, the end-to-end latency for the Resource strategy is typically higher than for the Latency strategy with the same *RF*.

The *Distributed Arithmetic* (DA) strategy implements the CMVM operation by explicitly decomposing it into an adder graph with only shift-and-add (or subtract) operations, where any bit-wise sparsity in the weights will be explicitly exploited. This strategy relies on an external library, DA4ML [114], for generating and optimizing the adder graph corresponding to the specific CMVM problems. Compared to the Latency strategy with an *RF* of one, the DA strategy usually has lower fabric resource utilization and latency. DSPs or other hardened multipliers will not be used due to the lack of explicit multiplications. However, the DA strategy in general does not support *RF*s greater than one since the generated adder graph effectively unrolls the entire CMVM operation. Hence, in case of moderately large CMVM problems consuming significant amount of hardened multipliers (e.g., DSPs), the user is encouraged to try the Latency or Resource strategy with a higher *RF* instead of the DA strategy.

Some layers, e.g., convolutional layers lowered to CMVM through the im2col [20] transformation, may perform identical CMVM operations multiple times on different inputs in one forward pass. In this case, the parallelism between CMVM operations is controlled by the *Parallelization Factor* (PF). The PF controls the number of CMVM operations that are performed in parallel, and it is independent of the strategy and *RF* used for the CMVM operation. A higher PF reduces the II of one layer at the expense of higher resource usage. Like *RF* in the Resource strategy, the PF chosen must fully divide the number of CMVM operations needed in one forward pass for the layer to avoid any remainder. The total II of one layer is determined by the PF (if available), *RF*, and strategy together.

Activations: Piecewise linear activations (e.g., ReLU, Leaky ReLU) are implemented using multiplexers. Other activations (e.g., tanh, sigmoid) are implemented as lookup tables⁴, which are populated at compile-time. The use of look-up tables guarantees constant and efficient calculation of the activation, regardless of its complexity. Given the precision of the layer input, hls4ml will generate a look-up table storing all possible output values of the activation function for each possible input value. In case the user-specified table size is smaller than the generated table, HLS4ML will drop the least significant bits from the input to fit the table size. Depending on the table size and the precision, look-up tables can be implemented as BRAM or LUTRAM, which is determined by the HLS compiler. For example, on AMD FPGAs, BRAM can be implemented as 18 bit wide by 1024 bit deep true dual-port RAM. Since the variable precision rarely exceeds 18 bits, one BRAM is used for each table of 1,024 elements. In the case of parallel accesses, the HLS compiler may duplicate the BRAM to ensure single-clock-cycle access. Empirically, we determine that activation look-up tables consume a small amount of resources and that

⁴Not to be confused with the LUT primitive on FPGAs.

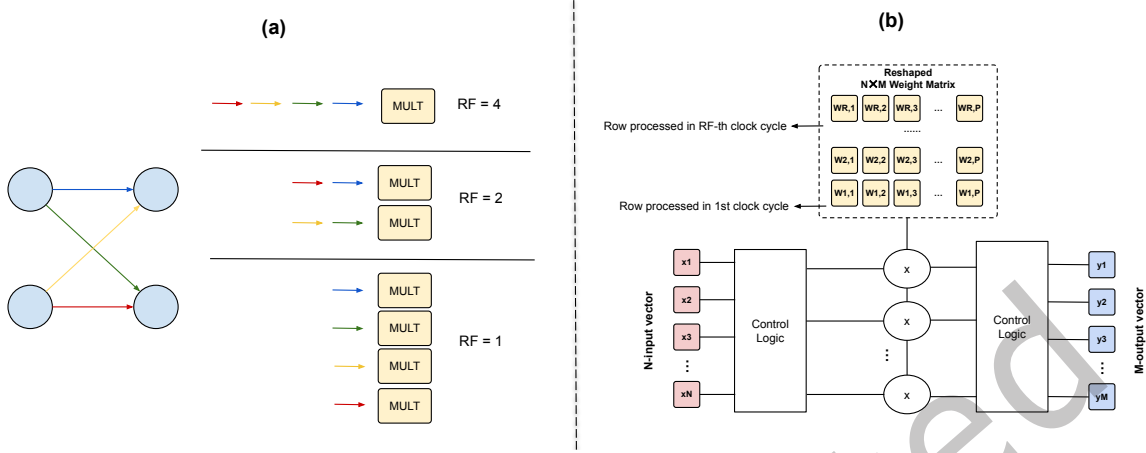


Fig. 3. (a) Illustration of the effect of different RF values for the outer product of two two-vectors. (b) Example CMVM in HLS4ML with the Resource strategy. Given a linear layer with N inputs, M outputs and reuse factor RF, there will be $P = \frac{M \cdot N}{RF}$ multipliers operating in parallel. In each clock cycle, the control logic selects P out of the N inputs and feeds them to the multipliers, with wrap around if $P > N$. The $N \times M$ kernel is reshaped and mapped to on-chip memories such that P elements can be accessed in parallel in each clock cycle. The products are accumulated accordingly at the precision specified to form the output.

most activations can be sufficiently approximated with a modest look-up table of ~ 2048 entries; though heavily quantized models may use even smaller look up tables.

Data flow, scheduling and I/O types: Models deployed with HLS4ML target *dataflow* architectures, in which each operation is mapped to a dedicated hardware function synthesized into a separate region of the FPGA fabric. This approach enables pipeline parallelism across the computation graph allowing for low latency and high throughput. In general, dataflow is achieved through directives such as the `#pragma HLS dataflow` in Vitis HLS or tasks in oneAPI. The exact scheduling of layers and data movement between them is, however, determined by the HLS compiler and depends on a number of factors, including the target FPGA platform, clock frequency/uncertainty, and model size. For example, increasing the clock frequency will often lead to higher latency, since the HLS compiler automatically inserts more pipeline stages to better achieve timing closure during place and route.

The data flow between layers is a crucial part of the design. HLS4ML supports two types of data transfer between layers: `io_parallel` and `io_stream` (example in Figure 4). The `io_parallel` setting directly wires the output of one layer to the input of the next layer, allowing for maximum throughput and minimum latency. This is the default data transfer type for HLS4ML, which is suitable for small models with moderate resource usage. On the other hand, `io_stream` uses FIFO buffers between layers. This is typically used with larger models where some layer requires a significant II, and the data processed by the previous layer need to be buffered before being consumed by the next layer. Depending on the buffer depth, the HLS compiler may elect to use shift registers or BRAM to implement the FIFOs. By default, HLS4ML uses a conservative depth for these buffers, which can result in unnecessary resource overheads. In the Vivado and Vitis back ends, a special optimizer is implemented to optimize the depth of these FIFOs by performing an RTL co-simulation of the model and recording the maximum occupation of each FIFO buffer [17]. The optimizer then sets the final depth for each FIFO to the maximum occupation plus one.

architecture more closely than the Quartus back end, where individual layers run independently as separate tasks, with handshaking and data transfers between tasks done through the streams implemented as pipes. Unlike Vivado/Vitis back ends, the Quartus and oneAPI back ends do not provide distinct strategy implementations for the CMVM operations and generally use a resource-optimized approach.

HLS4ML also provides a back end for CATAPULT HLS, developed in collaboration with Siemens EDA and guided by evolving industry requirements for edge ML applications. CATAPULT HLS is widely used for the development of ASICs, while also supporting FPGA designs, making it a versatile solution for synthesizing C++/SystemC specifications across various hardware targets. Both Latency and Resource strategies are supported, and the `io_type` can be configured as either `io_parallel` or `io_stream`. For streaming implementations, the Catapult back end supports bottom-up synthesis, leveraging Catapult’s Bottom-Up (BUP) flow to enable hierarchical and scalable hardware generation. This methodology enables individual model layers to be synthesized independently without requiring the entire model to be available. Each layer is synthesized into a separate RTL implementation and reusable Catapult library component. This divide-and-conquer approach simplifies design management, supports incremental synthesis (e.g., eliminating the need to re-synthesize the whole model when modifying a single layer), and enhances scalability by reducing memory requirements on the synthesis servers.

6.3 Accelerator back ends

The accelerator back ends are extensions of the IP core back ends, which enable end-to-end design-to-deployment flows for specific FPGA/SoC platforms. The accelerator extensions are available for the Vivado and Vitis back ends, with oneAPI currently under development. The Vivado accelerator back end embeds the generated IP core into a larger design containing an DMA core and other necessary utilities for host communication. Beyond IP integration, the back end also facilitates communication between the IP core and a system’s memory controller unit (MCU) or the host CPU via a custom device driver. Two different Vitis accelerator back ends are currently being integrated into HLS4ML. The first uses the *Vitis IP flow*, which closely resembles the Vivado accelerator back end in functionality, primarily targeting the AMD/Xilinx Zynq boards. The second one utilizes the *Vitis System Design Flow*, which enables the creation of complete applications targeting datacenter AMD/Xilinx Alveo accelerators. In this case, the accelerator back end generates a Vitis IP kernel along with a wrapper to facilitate data transfers between the accelerator and the host CPU. This approach allows compiling one or multiple instances of the resulting IP into a binary file that can be directly loaded onto the accelerator card from the host. Additionally, the back end produces multi-threaded host code to manage kernel execution and optimize data movement. This setup enables overlapping PCIe data transfers with kernel computation, effectively multiplexing communication and processing. In practice, this approach has achieved core occupancy levels of up to 96 % of the theoretical throughput, as estimated from the II and the kernel’s operating frequency. Multi-kernel and multi-accelerator configurations have demonstrated throughput exceeding 1.4 million inferences per second. HLS4ML’s generic design and interfaces are showcased by its integration with other FPGA shells, such as Coyote v2 [97], which also allows seamless deployment of applications on Alveo boards.

7 Community tools for model optimization and co-design

With the growing need for higher performance and accuracy, machine learning has shifted towards techniques such as quantization and pruning to efficiently deploy models on a variety of hardware. Such techniques can also be *hardware-aware*, in which the quantization precision or sparsity pattern is determined by the specific computation or memory model of the target device. Model-hardware co-design often yields the best performance with the lowest drop in accuracy. Following this trend, the community has proposed a growing number of co-design tools. We introduce some tools proposed by the community that are supported by HLS4ML.

7.1 QKeras

QKERAS is a widely used QAT library built on top of KERAS 2, developed by Google in collaboration with some of the HLS4ML authors [29]. QKERAS was the first QAT framework to be natively supported in HLS4ML. The library supports three quantization types: fixed-point integer, binary, and power-of-two quantization, and provides drop-in replacements for many KERAS 2 layers with the corresponding quantized version. To define a quantized model with QKERAS, one needs to replace the corresponding Keras layer with the corresponding QKERAS counterpart (e.g., Conv2D→QConv2D, Dense→QDense), and specify the target quantization type and precision. AutoQKeras, a hyperparameter tuning tool, may be used to automatically search for the best quantization precision for each layer in the model given a set of constraints on the model size, latency, and accuracy [29]. QKERAS has been adopted by various applications, including image classification with quantized CNNs [2], ternary neural networks for network traffic filtering [60], low-precision models for jet tagging at the Large Hadron Collider (LHC) [29, 86], and muon tracking [111].

7.2 High-granularity quantization

High-granularity quantization (HGQ) [113] is a QAT library built on top of KERAS that applies differentiable quantization to weights and activations at a sub-layer granularity. For models with unrolled CMVM operations, quantization granularity down to the per-parameter level can be used to exploit the FPGA's fine-grained parallelism. HGQ supports only fixed-point integer quantization, treating both binary and power-of-two quantization as special cases of the 1-bit quantization. Allowing learnable bit-widths to reach 0 bits automatically includes pruning as a special case of quantization. HGQ provides an accurate and differentiable resource usage estimation for the quantized model in the form of effective bit operations (EBOPs), a proxy for computational effort. This metric is used as a regularization term in the loss function, controlled by a hyperparameter β , which allows the user to control the trade-off between accuracy and resource usage during training. On models with unrolled CMVM operations, HGQ demonstrates significant advantages over other model compression techniques, with an on-chip LUT usage reduction of 50% to 95%, DSP usage reduction of 50% to 100%, and a latency reduction up to 80% with no inference performance degradation [112, 113]. There are two versions of HGQ: HGQ 1 based on KERAS 2, and HGQ 2 based on KERAS 3. Both libraries are tightly integrated with HLS4ML, and no user intervention is required to generate bit-exact HLS designs from the quantized models.

7.3 Distributed arithmetic

Distributed arithmetic is a technique for implementing fixed-point multipliers in hardware via shift-and-add operations. DA4ML [114] is a library that implements CMVM operations with an adder graph structure. The adder graph in DA4ML is optimized by a set of heuristics to minimize the number of adders weighted by their bit-width, including common sub-expression elimination (CSE). DA4ML targets unrolled CMVM operations, which constitute the dominant on-chip resource consumption for latency-critical models generated by HLS4ML. In contrast to other optimization techniques, DA4ML is an algorithm transformation at implementation time, usually taking only a few seconds, which does not change the model's output by a single bit relative to the corresponding fixed-point implementation. When applied to models trained with HGQ, DA4ML can achieve further LUT usage reduction of up to $\sim 1/3$ depending on the specific model, while completely eliminating the DSP usage [113, 114].

For some models that require the unrolling of large loops, especially of those contain many multiplication by zero, the HLS code generated directly with HLS4ML may be unsynthesizable, depending on the HLS compiler. They may, however, be synthesized with DA4ML [95, 112] due to its explicit loop unrolling and resource reduction. For each CMVM operation implemented with DA4ML, an accurate estimate of LUT usage is also provided. The DA4ML library is tightly integrated with HLS4ML, and setting the layer strategy to "distributed_arithmetic" in the HLS4ML configuration will enable the distributed arithmetic implementation for the corresponding layers.

7.4 DSP- and BRAM-aware pruning

Model pruning is an effective way to reduce the resource utilization of a model. However, the benefit of unstructured pruning is limited for designs where the multipliers are reused, and it is necessary to prune the weights in an organized manner aligned to the specific on-chip primitives used. The DSP- and BRAM-aware pruning algorithm [98] is implemented in a submodule of HLS4ML based on QKERAS with objectives constructed directly corresponding to reductions in on-chip primitive usage. In each case, the algorithm solves a Knapsack problem where weights in the model are assigned an importance value and a hardware cost. Given a certain resource capacity, the algorithm identifies which weights to prune. As a basic objective, the model can be optimized for sparsity itself in an unstructured approach. However, for a more structured approach, hardware-specific metrics are available. For FPGAs, these can be registers, DSPs, BRAM, or all of them simultaneously. For instance, all weights processed by a single DSP can be grouped and pruned together, and hence a DSP block can be removed from the design once a group of weights is pruned. Depending on the specific application, reductions in DSP usage by factors ranging from 2.2× to 12.2× and BRAM usage reductions by factors ranging from 1.4× to 5.2× with minimal accuracy degradation have been demonstrated [98].

7.5 Symbolic expression

HLS4ML includes a symbolic regression (SR) interface for creating HLS designs based on analytic expressions. SR aims to find a closed-form mathematical expression, containing operators such as $\exp(\cdot)$, $\sin(\cdot)$, $\cos(\cdot)$, that describe the mapping between given input and output data. These expressions can be efficiently implemented in hardware, especially if the functions are approximated with look-up tables, making them suitable for low-latency applications. Support for building efficient IPs implementing SR models is included in HLS4ML on top of the Vivado/Vitis back ends, leveraging the built-in code generation functionality. Specifically, the SR back end supports the translation of expressions represented as strings or SymPy objects, obtained through tools like `pySR` [33] or `SymbolNet` [119]. The functions within expressions can be implemented either using the native Xilinx HLS math library or approximated via LUTs, giving users an option to find the desired balance between accuracy, resource usage, and latency.

7.6 Surrogate models

The rapid generation of HLS designs for various models with different hardware configurations opens up the possibility of systematic studies of the design space. However, waiting for HLS synthesis results is often impractical and lengthy. An alternative is to use surrogate models, which can accurately predict the model resource utilization and latency. This can assist in making design choices by providing fast and accurate hardware resource estimates for different model parameter choices. One example is `RULE4ML` [96], an open-source tool that uses an MLP surrogate model to predict the resource usage of MLPs generated with HLS4ML. Trained on over 15,000 HLS synthesis results covering a wide range of MLP parameter space and HLS4ML design configurations, it achieves predictions within 10% of the true synthesis results in ~80% of test cases. In a related approach, `WA-HLS4ML`, over 100,000 models were generated, and a graph neural network, `lui-GNN`, was trained as the surrogate model [57].

7.7 Automated neural architecture and hardware co-design

Neural architecture co-design (NAC) [135] is a pipeline for neural architecture search and network compression in a two-stage approach based on the “Once-for-All” paradigm [19] to discover hardware efficient models. This approach consists of a global search stage that explores a wide range of architectures while considering hardware constraints, followed by a local search stage that fine-tunes and compresses the most promising candidates through QAT and pruning. Currently, the hardware cost is quantified using bit operations (BOPs), but integration of HLS4ML resource and latency estimates is underway. NAC has been demonstrated with Bragg peak finding in

materials science and jet classification in HEP, achieving models with improved accuracy, smaller latencies, or reduced resource utilization.

A recent advancement in this direction is MetaML-Pro [92], a co-optimization framework that automates design space exploration across neural network and hardware abstraction levels. It supports reusable optimization tasks, such as pruning, quantization, and scaling, and enables top-down and bottom-up information flow between software and hardware stages. By integrating tools like HLS4ML and leveraging Bayesian optimization, MetaML-Pro efficiently balances latency, accuracy, and resource usage, and has been demonstrated on jet tagging models for HEP. Complementary to MetaML-Pro, another recent framework extends co-design principles to incorporate trustworthiness by optimizing not only for accuracy and efficiency but also for uncertainty estimation [93]. It introduces Bayesian CNNs with Monte Carlo Dropout and automates trade-offs among predictive accuracy, computational cost, and model calibration, supporting use cases such as safety-critical inference on FPGAs.

7.8 SoC integration

The Embedded Scalable Platform (ESP) is an open-source research framework for designing and prototyping heterogeneous system-on-chip (SoC) architectures using a modular, tile-based approach [78]. It supports the automated integration of hardware accelerators, processors, memory tiles, and I/O components into an RTL SoC design. ESP integrates with HLS4ML to enable end-to-end deployment of machine learning models as hardware accelerators [50]. After HLS4ML generates a synthesizable hardware block from a neural network, ESP wraps it with standard interfaces and connects it to the rest of the SoC, automatically generating testbenches, drivers, and software applications for verification and experimentation. This combined workflow enables the design from a high-level ML model to complete system-level ASIC implementation. FPGA-based emulation and validation are also supported.

8 User-defined functionality

While HLS4ML supports a wide range of neural networks from a variety of libraries, users may wish to include custom operations in their designs. For layers that are not natively supported by HLS4ML, users need to provide their own HLS implementation. If these are of wider interest to the community, they can be fully integrated into and contributed to the tool. However, when full integration is not desired, HLS4ML can be extended via the `Extension API`, available for both the Keras 2 and PyTorch front ends, and all supported back ends. The `Extension API` allows users to leverage most of HLS4ML's rich infrastructure for other layers, optimizers, and HLS templates, while enabling them to implement the missing operators for their target applications. For this, the front end, IR, and back end must all be extended to support the new layer. On the front end side, this requires a custom layer handler function to parse the properties of the new layer. A class inheriting from the base layer class in the HLS4ML IR must also be defined. Finally, the back end requires Python templates for the configuration, function calls, as well as the custom HLS code of the new layer. These components are then registered with HLS4ML through the `Extension API`. Optionally, optimizers for the new layer can also be registered. A complete example can be found in the HLS4ML documentation online⁶. A practical example can be found in Reference [86], where within the context of an interaction network for jet tagging at CERN, the `Extension API` has been used to implement a custom Keras 2 projection layer, consisting of left- and right-multiplication by graph adjacency matrices. In addition to extending HLS4ML to support new layers, this mechanism could also be used to provide custom HLS implementations for existing layers, to e.g. optimize the design for a specific application or use different design techniques.

⁶<https://fastmachinelearning.org/hls4ml/advanced/extension.html>

9 Evaluation

Given the breadth of possible applications for HLS4ML, only a fraction of its capabilities can be demonstrated here. The initial performance studies for the different types of networks supported by HLS4ML can be found in the respective publications [1, 2, 40, 47, 69, 70, 74]. Additionally, prior works have shown acceleration of larger CNNs (ResNet- and UNet-like architectures) [17, 41] as well as large-scale, distributed models over a cluster of FPGAs [77, 116] with HLS4ML. Here, we instead present quantitative results obtained with the HLS4ML framework for a variety of benchmark models as new baselines for future works, with the aim of highlighting more recently added capabilities, such as the oneAPI and Catapult HLS back ends. As a community driven project, we also highlight the benefits of the evolving HLS4ML ecosystem. Therefore, results obtained using the HGQ quantization library and the new DA strategy are presented. These are post-route results of designs generated by HLS4ML targeting AMD Xilinx UltraScale+ FPGAs (XCVU9P and XCVU13P), or an Altera Agilex7 FPGA. We further highlight the wide range of applications enabled by HLS4ML by discussing models implemented on ASICs using the Catapult back end, and studies evaluating the performance of the HLS code generated by HLS4ML.

9.1 High-level feature jet tagger

The high-level jet tagger is a commonly used benchmark for low-latency neural networks on FPGAs [90]. The task is to classify jets (collimated particle showers at collider physics experiments), into five classes based on their originating particle. The inputs for each jet are 16 scalar values representing physics-motivated high-level features. The model architecture employed is an MLP from [40]. The results targeting an AMD XCVU9P FPGA are shown in Table 3.

All models use the `io_parallel` data transfer strategy. The models are quantized with HGQ and deployed with both the built-in Latency strategy and the DA strategy provided by DA4ML. For HGQ trained models, the results shown in different rows are quantized to different bit-widths, regularized by different β values. The results show major improvements when using HGQ over the previous baseline presented in [30], shown in the table labeled as QKERAS. The new DA strategy further reduces the resource usage and latency of the designs.

In addition, we add the maximum achieved clock frequency $F_{\max}$ for the models when targeting a 1 GHz clock period in Table 4 obtained for these designs with the default settings in Vivado and Vitis 2023.2 as a reference. The results show that the DA strategy may achieve lower latency and comparable or higher $F_{\max}$ compared to the Latency strategy. We further show results on a more resource-constrained AMD XC7A35T FPGA in Table 5. In the Latency strategy, the most accurate model failed to place due to insufficient resources, while the DA strategy successfully placed all models, with the most accurate model utilizing more than 90% of LUTs on the device. This demonstrates that HLS4ML also enables deployment of models on low-end, resource-constrained FPGAs. Compared to the larger FPGAs, the main difference is reflected in the latency and the clock frequency. Since the AMD XC7A35T is an FPGA of a lower speed grade, the HLS compiler inserts additional register stages to meet timing, leading to higher latency and resource consumption. Additionally, the absolute increase in resources, compared to the larger FPGA, can be attributed to logic duplication during place-and-route, again in order to achieve timing closure.

On the same task, we also processed a QKERAS trained model with 76.9% accuracy using the oneAPI back end, and evaluated its performance using oneAPI 2025.0 and QUARTUS 23.1. The results are given in Table 6. Given the model's small size, we continue using `io_parallel` for data transfer between the layers, and we fully unroll the layers. For this compiler, we use different compiler flags for different optimization targets. We found that the lowest latency can be obtained when requesting the lowest target frequency simultaneously with the latency optimization target. As this model has an II of one under this configuration, its throughput is proportional to the clock frequency given a continuous data stream. Therefore, it is possible to trade off between latency and throughput by adjusting the target frequency.

Table 3. Accuracy, resource consumption, latency, and initiation intervals (IIs) of the jet tagging models. Resources are reported after out of context place-and-route with an AMD XCVU9P FPGA. The clock period used is 5 ns for all designs shown, except for the design marked with *, which used 7 ns. All designs use no BRAM. The results for QKeras trained models are cited from [30] and for HGQ cited from [114].

Trainer	Strategy	Accuracy	Latency (cc)	DSP	LUT	FF	II
QKeras [98]*	Latency	76.3%	15 [105 ns]	5,504	175	3,036	2
QKeras [92]	Latency	76.1%	10 [50 ns]	13,042	70	N/A	1
HGQ	Latency	76.9%	5 (23.6 ns)	55 [0.8%]	13,303 [1.1%]	2,374 [0.1%]	1
HGQ	Latency	76.5%	5 (23.1 ns)	30 [0.4%]	6,715 [0.6%]	1,348 [0.1%]	1
HGQ	Latency	75.9%	3 (12.7 ns)	15 [0.2%]	3,044 [0.3%]	641 [0.0%]	1
HGQ	DA	76.9%	5 (23.4 ns)	0	11,978 [1.0%]	2,117 [0.1%]	1
HGQ	DA	76.5%	4 (18.0 ns)	0	6,067 [0.5%]	1,178 [0.0%]	1
HGQ	DA	75.9%	3 (12.1 ns)	0	2,891 [0.2%]	651 [0.0%]	1

Table 4. Accuracy, resource consumption, latency, and $F_{\max}$ of the jet tagging models trained in HGQ [113] when targeting 1 GHz clock. Resource reported are after out of context place-and-route with an AMD XCVU9P FPGA (part number: xcvu9p-flga2104-2L-e). All designs use no BRAM and have an II of one cycle.

Trainer	Strategy	$F_{\max}$	Accuracy	Latency (cc)	DSP	LUT	FF
HGQ	Latency	739 MHz	76.9%	37 (50.1 ns)	57 [0.8%]	14,624 [1.2%]	21,365 [0.9%]
HGQ	Latency	705 MHz	76.5%	31 (44.0 ns)	30 [0.4%]	7,656 [0.6%]	11,513 [0.5%]
HGQ	Latency	694 MHz	75.9%	30 (43.2 ns)	17 [0.2%]	3,765 [0.3%]	5,845 [0.2%]
HGQ	DA	713 MHz	76.9%	28 (39.3 ns)	0	12,465 [1.1%]	18,004 [0.8%]
HGQ	DA	719 MHz	76.5%	23 (32.0 ns)	0	6,297 [0.5%]	9,524 [0.4%]
HGQ	DA	756 MHz	75.9%	21 (27.8 ns)	0	2,886 [0.2%]	4,667 [0.2%]

Table 5. Accuracy, resource consumption, latency, and $F_{\max}$ of the jet tagging models trained in HGQ [113]. Resource reported are after out of context place-and-route with an AMD XC7A35T FPGA (part number: xc7a35t1csg325-2L). All designs use no BRAM and have an II of one cycle.

Trainer	Strategy	$F_{\max}$	Accuracy	Latency (cc)	DSP	LUT	FF
HGQ	Latency	-	76.9%	-	-	Place failed	-
HGQ	Latency	187 MHz	76.5%	53 (284.0 ns)	30 [33.3%]	11,091 [53.3%]	24,649 [59.3%]
HGQ	Latency	182 MHz	76.0%	45 (246.7 ns)	17 [18.9%]	5,565 [26.8%]	11,330 [27.2%]
HGQ	DA	172 MHz	76.9%	58 (337.4 ns)	0	19,467 [93.6%]	36,995 [88.9%]
HGQ	DA	185 MHz	76.5%	50 (270.1 ns)	0	10,692 [51.4%]	20,407 [49.1%]
HGQ	DA	176 MHz	75.9%	46 (261.1 ns)	0	5,455 [26.2%]	9,952 [23.9%]

9.2 Street View House Numbers classifier

Classification of the house numbers in images from the Street View House Numbers (SVHN) dataset [84] is a common benchmark for computer vision tasks. We show the results from [113] and [2], where both used the same model architecture as in [2]: a CNN with 3 convolutional layers, 3 max pooling layers, and 2 fully connected layers. The detailed description of the architecture can be found in both works. For HGQ trained models, the results shown in different rows are quantized to different bit-widths, regularized by different β values. The results

Table 6. Resource consumption, latency, and $F_{\max}$ of the jet tagging QKERAS models using ONEAPI. All designs have an accuracy of 76.9%. Resources reported are after out of context place-and-route, as reported by QUARTUS, with an Altera Agilex7 FPGA (part number: AGFB014R24A2E2V). The latency is measured first input to first output, reported both in clock cycles and in nanoseconds if running at $F_{\max}$. The optimization target is an option given to the icpx compiler, unrelated to the HLS4ML strategy. The default option, obtained by not explicitly specifying an optimization target, optimizes for maximum frequency, and turns on hyper-optimized handshaking. All designs shown have negligible MLAB, RAM, and DSP usage (less than 0.01%) and an II of one.

Opt. Target	Target $F_{\max}$	$F_{\max}$	Latency (cc)	ALM	ALUT	FF
Latency	200 MHz	341 MHz	21 [62 ns]	11,434 [2.3%]	21,471 [2.2%]	8,864 [0.45%]
Latency	480 MHz	614 MHz	50 [81 ns]	12,085 [2.5%]	22,551 [2.3%]	31,243 [1.6%]
Default	480 MHz	782 MHz	90 [115 ns]	13,576 [2.8%]	24,319 [2.5%]	40,051 [2.1%]

are shown in Table 7. All designs shown are using $PF = 1$ (each convolution kernel is applied once per clock cycle) and `io_stream`. Compared to the previous results obtained in [2] with HLS4ML, the new results use significantly fewer on-chip resources while retaining the same accuracy. As in the previous section, we find that the DA strategy offers improved performance compared to the Latency one, most notably by eliminating all DSP usage, while reductions in LUTs, FFs, and latency are less pronounced. However, this advantage might not transfer to cases where high variable precision is required, and all cases with an RF larger than one, which are not supported in the DA strategy.

Table 7. Accuracy, resource usage, and latency of the SVHN classifier models. Reported resource usage after out of context place-and-route with an AMD XCVU9P FPGA. The clock period used is 5 ns. The results for QKERAS trained models are cited from [2], and the HGQ trained models are cited from [113].

Trainer	Strategy	Accuracy	Latency (cc)	DSP	LUT (k)	FF (k)	BRAM	II (cc)
QKERAS	Latency	94.4%	1,035 [5.2 μ s]	174 [2.54%]	111 [9.4%]	33 [1.4%]	67.0 [3.1%]	1,030
QKERAS	Latency	88.8%	1,059 [5.3 μ s]	72 [1.05%]	48 [4.1%]	15 [0.63%]	32.5 [1.5%]	1,029
HGQ	Latency	93.9%	1,050 [5.3 μ s]	58 [0.85%]	69 [5.8%]	28 [1.2%]	32.0 [1.5%]	1,029
HGQ	Latency	93.1%	1,061 [5.3 μ s]	30 [0.44%]	47 [4.0%]	21 [0.89%]	28.0 [1.3%]	1,029
HGQ	Latency	91.9%	1,058 [5.3 μ s]	15 [0.22%]	40 [3.4%]	18 [0.76%]	23.5 [1.1%]	1,029
HGQ	DA	93.9%	1,045 [5.2 μ s]	0	53 [4.5%]	20 [0.85%]	32.0 [1.5%]	1,029
HGQ	DA	93.1%	1,045 [5.2 μ s]	0	37 [3.1%]	15 [0.63%]	28.0 [1.3%]	1,029
HGQ	DA	91.9%	1,045 [5.2 μ s]	0	31 [2.6%]	14 [0.59%]	23.5 [1.1%]	1,029

9.3 Particle-based jet tagger

The particle-based jet tagger is a more challenging benchmark for low-latency neural networks on FPGAs [90]. Similar to the high-level jet tagger, the task is to classify jets into five classes based on their originating particle. However, instead of using the 16 high-level features, which are unlikely to be available in real-time, the raw particle information is fed into the model. Up to 150 particles are available for each jet, ordered by their transverse momentum. Each particle has 16 features, and up to 64 particles are used in the model (zero-padded if less than 64 particles are available). The model architecture is an MLP-Mixer tailored for this task, and the detailed description can be found in [112]. The results are reproduced following the same setup as in [112] with updated template functions and the latest HLS4ML version. The results are shown in Table 8. The models shown on different rows are quantized to different bit-widths, regularized by different β values. The `io_parallel` configuration is used

for all models, and only the DA strategy is used, as the Latency strategy fails to converge in timing and to meet the initiation interval constraints. This is likely due to the overly large, but highly sparse kernel matrices used in the MLP-Mixer architecture, where the HLS compiler’s heuristic failed to implement the design properly.

Table 8. Accuracy, resource usage, and latency of the particle-based classifier models in HGQ. Reported resource usage after out of context place-and-route with an AMD XCVU13P FPGA. The clock period used is 5 ns. All HGQ trained models are using the MLP-Mixer architecture from [112]. Models without DA failed to meet timing and II constraints. All designs use no DSPs or BRAM and have an II of one cycle.

Trainer	Strategy	Accuracy	Latency (cc)	LUT (k)	FF (k)
HGQ	DA	81.4%	13 [65 ns]	126 [7.3%]	26 [0.76%]
HGQ	DA	81.0%	13 [65 ns]	68 [3.9%]	13 [0.38%]
HGQ	DA	80.3%	12 [60 ns]	51 [3.0%]	11 [0.33%]

9.4 MNIST classification

Classification of images of handwritten digits from the Modified National Institute of Standards and Technology (MNIST) database [36] is a common benchmark for computer vision tasks. The task is to classify handwritten digits from 0 to 9. The model architecture employed a single layer MLP with 128 hidden units and 10 output units. We trained three models with different quantization levels with HGQ, all using the `io_parallel` approach for data transfer between layers, and the results are shown in Table 9. Only the DA strategy can be applied to these models, as when using the Latency strategy, the sparse 768×128 kernel matrix fails to unroll properly, leading to HLS synthesis failures.

Table 9. Accuracy, resource usage, latency, and initiation intervals of the MNIST classifier models in HGQ [113]. Reported resource usage after out of context place-and-route with an AMD XCVU13P FPGA. The clock period used is 5 ns. All HGQ trained models are using a single layer MLP. Models without DA failed to synthesize. All designs use no DSPs or BRAM and have an II of one cycle.

Trainer	Strategy	Accuracy	Latency (cc)	LUT	FF
HGQ	DA	97.7%	2 [10 ns]	9671 [0.56%]	2,039 [0.06%]
HGQ	DA	97.3%	2 [10 ns]	6890 [0.40%]	1,672 [0.05%]
HGQ	DA	97.0%	2 [10 ns]	5040 [0.29%]	1,349 [0.04%]

9.5 Smart pixels

In [89, 136], we demonstrated the implementation of a neural network-based filter for low-momentum charged particle tracks in “smart pixel” sensors, targeting on-chip data reduction to cope with the even higher data rates expected in future HEP experiments. Using the Catapult back end, we translated a compact quantized classifier (~1,200 trainable parameters) from QKERAS into synthesizable C++ and generated fully combinatorial RTL code for integration in 28 nm CMOS. The design was implemented in conjunction with the analog front end, using less than $1 \mu\text{W} / \text{pixel}$ for digital logic within a total power budget below $1 \text{ W}/\text{cm}^2$. The HLS-driven flow enabled faster design cycles, efficient exploration of precision and parallelism, and ultimately achieved a bandwidth reduction between 54.4% and 75.4%.

Instead of transmitting full pixel cluster information, we further aim to compress the data into a small number of variables that describe the kinematics of the traversing particle. Using a mixture density network implemented

directly in the pixel readout electronics, we can predict the track angle and hit position along with the associated uncertainties. This additional layer of intelligent data reduction enables even lower bandwidth and downstream computational complexity while preserving critical physics information. The most recent model is a five-layer quantized network, which in HLS4ML is implemented as a pipeline with an initiation interval of one, a latency of two clock cycles, using the `io_parallel` and Latency strategies. The model consists of two convolutional layers followed by three dense layers (~3,000 trainable parameters). The first convolutional layer is implemented as a depthwise-separable convolution to minimize the number of parameters and reduce computation, which was added to the Catapult back end in HLS4ML for this project. Using 2D convolutions, this model has an estimated area of 0.81 mm^2 in 28 nm CMOS, while a 1D convolutional variant achieves comparable functionality with a significantly reduced area of 0.39 mm^2 . Ongoing design space exploration is focused on further minimizing area while preserving predictive performance and hardware feasibility.

9.6 Quality of HLS4ML results

While the ease of use of HLS frameworks like HLS4ML is a significant benefit, the quality of results is a critical factor for adoption. Comparing the resource usage, latency, and throughput of HLS4ML generated designs against hand-mapped Verilog designs for the same models, the HLS-synthesized designs can benefit from more sophisticated constant folding in the HLS compiler compared with the FPGA vendor's standard synthesis tools, the exploitation of carefully constructed Verilog codes that better invoke the back end tools, and other optimizations [71, 72]. The HLS4ML flow can achieve superior performance compared to hand-mapped designs in some cases, while being up to $2\times$ worse in the worst case. Similar results are demonstrated in [104] for automated synthesis flows targeting embedded AI engines (AIEs). When comparing HLS4ML designs automatically mapped to the FPGA fabric to software designs mapped automatically to the VLIW processors embedded in some AMD Versal devices, the HLS4ML designs were generally superior to the AIE implementations in resource usage, latency, and throughput, while being up to $2\times$ worse in the worst case.

10 Applications of HLS4ML

HLS4ML was originally developed for HEP applications at CERN and has since been widely adopted for a variety of use cases. The ATLAS and CMS detectors [31, 32] at the CERN LHC [44] deploy a data filtering system on FPGAs that decides in real time which collisions to store persistently, reducing data volumes from approximately 100 TB/s to about 10 GB/s with a typical latency constraint of $\sim 10 \mu\text{s}$. As of now, three algorithms have been deployed using HLS4ML. The first is an MLP to assign muon transverse momentum and displacement in triggers for unconventional signatures [58]. The AXOL1TL [25, 27] and CICADA [26, 28] algorithms are variational and convolutional autoencoders, respectively, trained to detect anomalies [52] in the detector data with different levels of preprocessing. Similar applications with HLS4ML are being considered for other HEP experiments and related applications [49, 56, 79].

The need for real-time data analysis makes the deployment of ML models on FPGAs desirable in other scientific environments as well. For example, HLS4ML was used to demonstrate that CNN models deployed on FPGAs could meet latency requirements that would enable self-triggering of radio antennas for cosmic ray detection [39]. Similarly, the necessity of real-time control of sensitive systems makes these techniques suitable for quantum computing applications, where HLS4ML has been used in the development of MLP models for the control [14] and read out [37] of qubits. As an example of an application in accelerator control [109], a demonstrator system was designed using HLS4ML employing an Intel Arria 10 FPGA-based board to monitor and correctly attribute beam losses in the Fermilab Recycler Ring [105]. Similarly, for active feedback control in magnetic confinement fusion devices an inference latency of $7.7 \mu\text{s}$ and end-to-end latency of $17.6 \mu\text{s}$ was achieved, enabling the tracking of magnetohydrodynamic modes in the plasma at over 100 kfps [134]. Potential applications also extend to radiation

safety, where a compact, light-weight, low-power radiation detector has been developed in which an MLP model is deployed on an Artix-7 Series FPGA using HLS4ML [82].

HLS4ML has also expanded out from scientific computing into diverse domains including automotive and space applications. It has been used to prototype a quantized CNN for semantic segmentation in self-driving cars [47], achieving a latency of ~ 5 ms while using less than 30% of the available resources on a Xilinx ZCU102 board. Power efficiency constraints are a key concern for deploying ML models on satellites and other spacecraft. HLS4ML is being used in the development of models for environmental monitoring of the earth using satellites [108]. In another signal processing application, HLS4ML has enabled the real-time analysis of microwave sensing data for food contamination using an MLP [130], achieving a latency of $27 \mu\text{s}$ on an AMD/Xilinx Kria K26 FPGA. HLS4ML was used to deploy a combined CNN and RNN model (C-RNN) for accurate classification of artificial surface textures with a latency of $6.21 \mu\text{s}$ [117]. Demonstrating its usefulness in biomedical applications, HLS4ML was also used to prototype a 1D-CNN for arrhythmia classification from ECG waves on a system-on-chip (SoC), with a power consumption of 1.655 W [80]. Another example is the classification of lymphocyte cells, where HLS4ML was used to demonstrate a $12\times$ reduction in inference latency compared to GPUs when deploying the classifier on a Euresys frame grabber FPGA [67]. HLS4ML was used to accelerate a ternary model for detecting malicious executables in network packets with a latency of 44 ns , while maintaining the 100 Gbps network throughput [60]. In a speech recognition application, a 1D CNN model was deployed using HLS4ML on a Pynq-Z2 FPGA, achieving a performance improvement of $6\times$ to $12\times$ compared to GPUs and CPUs [35]. Given the growing community, use cases, and contributions, continued development and support of HLS4ML as an open-source, high-performance and modular platform for neural network acceleration on FPGAs is imperative.

11 Challenges and limitations

HLS4ML's place in the ecosystem as an interface between a growing set of training libraries for deep learning models and HLS tools presents a variety of challenges for its continued development. The large number of front and back ends already available or in active development will further increase the maintenance burden and requires increased community engagement. The heterogeneous support of features in the different front and back ends outlined in Section 3 is therefore likely to persist, depending on the level of community interest in the different tools. This is exacerbated by the fact that novel implementations derived using HLS4ML in the context of research projects are not always easy to generalize and contribute back to the tool. This limits the usability of HLS4ML for some tasks, especially if the user is restricted to use one of the lesser developed back ends.

Problems can also arise from HLS tools, especially for larger models where the complexity might overwhelm the HLS compiler, leading to long, or in extreme cases, failed HLS synthesis. Given the integration of HLS4ML with a large number of external tools and products, maintaining full functionality of the tool and high performance requires extensive validation effort. Different HLS compiler versions frequently differ (e.g. supported pragmas may change), and changes in performance for the same HLS code between versions are common. This requires regular adjustments and reoptimization of the HLS templates in HLS4ML. In extreme cases, HLS4ML has to react to the discontinuation of whole tools, such as support for Altera FPGAs being dropped from the Intel ONEAPI compiler starting with version 2025.1.

12 Related work

The acceleration of ML/AI inference on FPGAs is a rapidly evolving field, with numerous proprietary and open-source solutions proposed. Broadly, FPGA neural network accelerators can be classified as (1) software-controllable, overlay-based or (2) custom hardware generators [18, 91]. Overlay architectures implement a fixed microarchitecture in FPGA fabric, which is controllable from software similar to CPUs and GPUs. Conversely, custom hardware accelerators generate optimized model-specific designs, typically by creating dedicated hardware

implementations for individual layers or operators in the model. In this section, we provide a brief overview of the existing tools and their features, focusing mainly on platforms with custom hardware generation, since HLS4ML falls into this category. More in-depth overviews of neural network acceleration on FPGAs, including overlay architectures, can be found in surveys [18, 91].

Table 10. Comparison of different tools for the translation of ML models to FPGA designs.

Framework	Multiple ML frameworks	Multiple hardware vendors	Arbitrary Quantization	CNN	RNN	Transformers	Open-source	Maintained
Vitis AI [9]	☑	☐	☐	☑	☑	☑	☑	☑
Intel FPGA AI Suite [66]	☑	☐	☐	☑	☑	☑	☐	☑
EdgeCortex MERA [42]	☑	☐	☐	☑	☑	☑	☑	☑
FP-DNN [53]	☐	☑	☑	☑	☑	☐	☐	N/A
DeepBurning [133]	☐	☑	☐	☑	☑	☐	☑	☐
DeepHLS [99]	☐	☑	☑	☑	☐	☐	☑	☐
PipeCNN [131]	☐	☑	☑	☑	☐	☐	☑	☐
DnnWeaver [103]	☐	☑	☐	☑	☐	☐	☑	☐
NNGen [115]	☑	☑	☑	☑	☐	☐	☑	☐
fpgaConvNet [126]	☑	☐	☑	☑	☐	☐	☑	⊕
HPIPE/H2PIPE [55, 110]	☐	☑	☑	☑	☐	☐	☐	N/A
MASE [22, 137]	☐	☑	☑	☑	☐	☑	☑	☑
cgra4ml [4]	☐	☑	☑	☑	☐	☑	☑	☑
chisel4ml [129]	☑	☑	☑	☑	☐	☐	☑	☑
tinyHLS [61]	☐	☑	⊕	⊕	☐	☐	☑	☑
FINN [15, 123]	☑	☐	☑	☑	☐	☑	☑	☑
HLS4ML [40]	☑	☑	☑	☑	☑	⊕	☑	☑

☑: Supported; ⊕: Partially supported; ☐: Not supported.

First group: commercial products; second group: research projects.

“Multiple ML frameworks” corresponds to a framework being able to ingest models from multiple different frameworks; however, if a framework supports ONNX/QONNX, it automatically supports multiple frameworks, since KERAS 2 and PYTORCH can be exported to ONNX. “Arbitrary quantization” refers to the ability to accelerate arbitrarily quantized models; rather than supporting only fixed quantization strategies (e.g., INT8, FP16).

Table 10 summarizes the projects described in this section and their capabilities. Commercial platforms include AMD VITIS AI [9] and Intel FPGA AI SUITE [66], both of which can convert neural network models from multiple deep learning frameworks. The MERA [42] compiler from EdgeCortex converts models from PYTORCH, TENSORFLOW, and ONNX into dataflow graphs that can be deployed on their dynamic neural architecture IP cores. However, neither framework extends to multiple hardware vendors nor supports arbitrary quantization.

On the research side, early platforms include FP-DNN [53] and DEEPBURNING [133], which use RTL to generate hardware designs, making them portable across platforms. HLS-based platforms include DEEPHLS [99] and PIPECNN [131] that support MLP and CNN models. Neither of these four platforms support multiple front ends, and to our knowledge, none are actively maintained. FPGA CONVNET [125, 127, 128] focuses specifically on translating large-scale CNNs defined in PYTORCH or ONNX into HLS code for VIVADO HLS. Similarly, HPIPE provides a workflow to create optimized CNN designs starting from TENSORFLOW models for various FPGAs using RTL design. Both frameworks include support for arbitrary, per-layer quantization. To support even larger models, H2PIPE [55] extends HPIPE with designs that utilize off-chip memory. For even larger models, such as transformers, two notable platforms are MASE [22, 137] and CGRA4ML [4]. MASE [22, 137] supports quantization and conversion of PYTORCH models into FPGA designs using templates in SystemVerilog, making it portable across hardware platforms. Notably, it directly enables QAT, with a strong focus on transformer models and

LLMs. Similarly, CGRA4ML [4] focuses on accelerating large neural networks by reusing processing units across layers, including support for MLPs, CNNs, and transformers. Unlike HLS4ML, CGRA4ML does not generate dataflow designs and stores weights in off-chip memory. Neither MASE nor CGRA4ML support multiple front ends: MASE is compatible with PyTorch and CGRA4ML with QKeras. CHISEL4ML [129] enables the translation of MLPs and CNNs from QONNX into fully parallelized, dataflow designs supporting arbitrary quantization via the Chisel hardware description language. In general, performance comparable to that of HLS4ML was reported [129]. On the other hand, TINYHLS [61] generates Verilog code enabling MLPs and one-dimensional CNNs defined in Keras 2 to be deployed on FPGAs. FINN [15, 123] is an open-source project by AMD/Xilinx that allows translating models defined in Brevitas or QONNX into hardware designs targeting AMD/Xilinx FPGAs. Like HLS4ML, it targets a dataflow architecture with fine-grained quantization control for low-latency, high-throughput application, and supports a wide range of neural network architectures.

13 Conclusions and outlook

FPGAs are uniquely suited for efficient and flexible deployment of neural networks in low-latency, low-power environments. However, traditional implementations of these models in RTL are complex and require specialized knowledge. A large number of tools and platforms aim to aid users in the translation of their models into FPGA designs, removing the need to re-implement common building blocks. Among these, HLS4ML is an open-source platform that stands out for its modular design and extensible nature. HLS4ML supports all major deep learning libraries and different HLS compilers, enabling deployment across a wide range of FPGA vendors as well as ASIC design via integration with CATAPULT HLS. Furthermore, HLS4ML supports a wide and growing range of model architectures and provides a convenient interface to include custom layers in the design. The FPGA designs available in HLS4ML are easily customizable for different objectives, such as latency or resource usage. Hardware-software co-design is crucial for achieving accurate and high-performance designs, with pruning and QAT as the most common techniques. HLS4ML supports models trained with most common QAT libraries and has integrated support for hardware-aware compression techniques. Thus, HLS4ML plays a central role in a growing ecosystem for optimized deployment of neural networks on FPGAs. Additionally, it can act as a platform that supports research in novel model-hardware co-design techniques.

HLS4ML is developed by an active, diverse community of researchers, continuously expanding its capabilities and features. At its core, the HLS4ML community strives to keep the platform modular and extensible, continuously adding new front and back ends. For example, a Flax/JAX front end and a SmartHLS compiler back end targeting Microchip PolarFire FPGAs are in development. In parallel, to support NanoXplore radiation-tolerant FPGAs, the Catapult HLS library will be extended, or alternatively, integration with the Panda Bambu HLS toolchain [46] will be added to the back ends. Significant emphasis is being placed on supporting and developing novel co-design libraries to maximize performance while minimizing resource and latency requirements. On the hardware design side, support for transformer architectures are active area of development, with a first implementation being available via the HGQ 2 framework. A variation of the transformer architecture optimized for computational efficiency on point cloud data using locality sensitive hashing (HEPT) [81] is also being developed in HLS4ML. Another promising direction for future development is the integration of AMD AI engines, and HLS4ML may take advantage of this architecture alongside its existing support for FPGA designs. Following HLS4ML's recent integration with Coyote [75, 97], an open-source shell with rich networking services and collective communication [59], future work may extend HLS4ML to support distributed inference, by leveraging these services. Such distributed inference would be particularly suitable for larger models that cannot fit on a single FPGA, e.g., transformers.

Acknowledgments

We acknowledge the importance of the Fast Machine Learning community for the development of this project. We especially thank Paolo D’Alberto, Duen-Jeng (DJ) Wang and Luciano Lavagno from AMD, as well as Yaman Umuroglu, Michaela Blott, and the overall FINN team, for helpful insights and facilitating technical discussions. This work was partly supported by the Accelerated AI Algorithms for Data-Driven Discovery (A3D3) Institute under U.S. National Science Foundation (NSF) Grant No. PHY-2117997. This work was supported by FermiForward Discovery Group, LLC under Contract No. 89243024CSC000002 with the U.S. Department of Energy (DOE), Office of Science, Office of High Energy Physics. Work done by Imperial College is funded by the Science and Technology Facilities Council (STFC) grant ST/W000636/1 and EPSRC (grant numbers UKRI256, EP/V028251/1, EP/N031768/1, EP/S030069/1, and EP/X036006/1). Enrico Lupi, Dimitrios Danopoulos, and Vladimir Lončar are funded by the Eric & Wendy Schmidt Fund for Strategic Innovation through the CERN Next Generation Triggers project under grant agreement number SIF-2023-004. Nicolò Ghielmetti is supported by the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (Grant Agreement No. 101135358). This work was partially supported by DOE ASRSP GCFA Grant ID: SP0062070 and NSF POSE Phase II Award 2303700. Dylan Rankin is supported by the DOE, Office of Science, Office of High Energy Physics Early Career Research program under Award No. DE-SC0025324. Javier Duarte is additionally supported by the Research Corporation for Science Advancement (RCSA) under grant #CS-CSA-2023-109, Alfred P. Sloan Foundation under grant #FG-2023-20452, and DOE, Office of Science, Office of High Energy Physics Early Career Research program under Award No. DE-SC0021187. Santosh Parajuli and Mark Neubauer are additionally supported by the U.S. Department of Energy, Office of Science, Office of High Energy Physics, under contract number DE-SC0023365. Chang Sun is partially supported by the NSF ACCESS Grant number PHY240298, DOE, Office of Science, Office of High Energy Physics grant under Award No. DE-SC0011925. Jennifer Ngadiuba and Chang Sun are partially supported by the DOE Office of Science, Office of High Energy Physics “Designing efficient edge AI with physics phenomena” Project (DE-FOA-0002705). Thea K. Aarrestad is supported by the Swiss National Science Foundation Grant No. PZ00P2_201594. Ryan Forelli gratefully acknowledges support from the Ryan Fellowship and the International Institute for Nanotechnology at Northwestern University. Benjamin Ramhorst gratefully acknowledges AMD for the donation of the Heterogeneous Accelerated Compute Cluster (HACC) at ETH Zurich, which was partially used for the development and testing of the work presented in this paper.

References

- [1] Georges Aad et al. 2021. Artificial Neural Networks on FPGAs for Real-Time Energy Reconstruction of the ATLAS LAr Calorimeters. *Comput. Softw. Big Sci.* 5 (2021), 19. doi:10.1007/s41781-021-00066-y
- [2] Thea Aarrestad et al. 2021. Fast convolutional neural networks on FPGAs with hls4ml. *Mach. Learn. Sci. Tech.* 2 (2021), 045015. doi:10.1088/2632-2153/ac0ea1 arXiv:2101.05108 [cs.LG]
- [3] Martin Abadi et al. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://www.tensorflow.org/> Software available from tensorflow.org.
- [4] G Abarajithan, Zhenghua Ma, Zepeng Li, Shrideep Koparkar, Ravidu Munasinghe, Francesco Restuccia, and Ryan Kastner. 2024. CGRA4ML: A Framework to Implement Modern Neural Networks for Scientific Edge Computing. (2024). arXiv:2408.15561 [cs.AR]
- [5] AMD. 2024. *Developing Vitis Kernels and Applications*. Retrieved May 30, 2025 from <https://docs.amd.com/r/en-US/ug1700-vitis-accelerated-data-center/Developing-Vitis-Kernels-and-Applications>
- [6] AMD. 2024. *Vitis High-Level Synthesis User Guide (UG1399)*. Retrieved May 1, 2025 from <https://docs.amd.com/r/en-US/ug1399-vitis-hls/Introduction>
- [7] AMD. 2025. *ABB Robotics Powers Collaborative Robots with AMD*. Retrieved May 6, 2025 from <https://www.amd.com/en/resources/case-studies/abb-robotics.html>
- [8] AMD. 2025. *Smart Camera*. Retrieved May 6, 2025 from https://xilinx.github.io/kria-apps-docs/kv260/2022.1/build/html/docs/smartcamera/smartcamera_landing.html

- [9] AMD/Xilinx. 2024. Vitis AI Development Platform. <https://www.xilinx.com/products/design-tools/vitis/vitis-ai.html>. Accessed April 23, 2025.
- [10] Marta Andronic, Jiawen Li, and George A. Constantinides. 2025. PolyLUT: Ultra-low Latency Polynomial Inference with Hardware-Aware Structured Pruning. arXiv:2501.08043 [cs.LG] <https://arxiv.org/abs/2501.08043>
- [11] Jason Ansel et al. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 929. doi:10.1145/3620665.3640366
- [12] Manish Arora, Shawn Bohrer, Omer Yoachimik, Cody Doucette, Alex Forster, and Nick Wood. 2024. *How Cloudflare auto-mitigated world record 3.8 Tbps DDoS attack*. Retrieved May 6, 2025 from <https://blog.cloudflare.com/how-cloudflare-auto-mitigated-world-record-3-8-tbps-ddos-attack/>
- [13] Junjie Bai, Fang Lu, Ke Zhang, et al. 2019. ONNX: Open Neural Network Exchange. <https://github.com/onnx/onnx>.
- [14] Madhav Narayan Bhat, Marco Russo, Luca P. Carloni, Giuseppe Di Guglielmo, Farah Fahim, Andy C. Y. Li, and Gabriel N. Perdue. 2025. Machine learning for arbitrary single-qubit rotations on an embedded device. *Quantum Machine Intelligence* 7, 1 (2025), 8. doi:10.1007/s42484-024-00214-8 arXiv:2411.13037 [quant-ph]
- [15] Michaela Blott, Thomas B Preußer, Nicholas J Fraser, Giulio Gambardella, Kenneth O'brien, Yaman Umuroglu, Miriam Leeser, and Kees Vissers. 2018. FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Trans. Reconfigurable Technol. Syst.* 11 (2018), 1.
- [16] ANDREW D. BOOTH. 1951. A SIGNED BINARY MULTIPLICATION TECHNIQUE. *The Quarterly Journal of Mechanics and Applied Mathematics* 4, 2 (01 1951), 236–240. doi:10.1093/qjmam/4.2.236 arXiv:<https://academic.oup.com/qjmam/article-pdf/4/2/236/5301697/4-2-236.pdf>
- [17] Hendrik Borrás, Giuseppe Di Guglielmo, Javier Duarte, Nicolò Ghielmetti, Ben Hawks, Scott Hauck, Shih-Chieh Hsu, Ryan Kastner, Jason Liang, Andres Meza, Jules Muhizi, Tai Nguyen, Rushil Roy, Nhan Tran, Yaman Umuroglu, Olivia Weng, Aidan Yokuda, and Michaela Blott. 2022. Open-source FPGA-ML codesign for the MLPerf Tiny Benchmark. In *3rd Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware (MLBench) at 5th Conference on Machine Learning and Systems (MLSys)*. arXiv:2206.11791 [cs.LG]
- [18] Andrew Boutros, Aman Arora, and Vaughn Betz. 2025. Field-Programmable Gate Array Architecture for Deep Learning: Survey and Future Directions. *Proc. IEEE* 113, 7 (2025), 613–639. doi:10.1109/JPROC.2025.3623023
- [19] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. 2020. Once for All: Train One Network and Specialize it for Efficient Deployment. In *International Conference on Learning Representations*. arXiv:1908.09791 <https://openreview.net/forum?id=HylxE1HKwS>
- [20] Kumar Chellapilla, Sidd Puri, and Patrice Simard. 2006. High Performance Convolutional Neural Networks for Document Processing. In *Tenth International Workshop on Frontiers in Handwriting Recognition*, Guy Lorette (Ed.). Université de Rennes 1, Suvisoft, La Baule (France). <https://inria.hal.science/inria-00112631> <http://www.suvisoft.com>.
- [21] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. 2024. A Survey on Deep Neural Network Pruning: Taxonomy, Comparison, Analysis, and Recommendations. *IEEE Trans. Pattern Anal. Mach. Intell.* 46, 12 (Dec. 2024), 10558–10578. doi:10.1109/TPAMI.2024.3447085
- [22] Jianyi Cheng, Cheng Zhang, Zhewen Yu, Christos-Savvas Bouganis, George A. Constantinides, and Yiren Zhao. 2024. A Dataflow Compiler for Efficient LLM Inference using Custom Microscaling Formats. arXiv:2307.15517 [cs.AR] <https://arxiv.org/abs/2307.15517>
- [23] François Chollet et al. 2015. Keras. <https://keras.io>.
- [24] Eric S. Chung et al. 2018. Serving DNNs in Real Time at Datacenter Scale with Project Brainwave. *IEEE Micro* 38, 2 (2018), 8–20. doi:10.1109/MM.2018.022071131
- [25] CMS Collaboration. 2023. *Anomaly Detection in the CMS Global Trigger Test Crate for Run 3*. CMS Detector Performance Summary CMS-DP-2023-079. <https://cds.cern.ch/record/2876546>
- [26] CMS Collaboration. 2023. *Level-1 Trigger Calorimeter Image Convolutional Anomaly Detection Algorithm*. CMS Detector Performance Summary CMS-DP-2023-086. <https://cds.cern.ch/record/2879816>
- [27] CMS Collaboration. 2024. *2024 Data Collected with AXOLITL Anomaly Detection at the CMS Level-1 Trigger*. CMS Detector Performance Summary CMS-DP-2024-059. <https://cds.cern.ch/record/2904695>
- [28] CMS Collaboration. 2024. *Model-Independent Real-Time Anomaly Detection at the CMS Level-1 Calorimeter Trigger with CICADA*. CMS Detector Performance Summary CMS-DP-2024-121. <https://cds.cern.ch/record/2917884>
- [29] Claudionor N. Coelho, Aki Kuusela, Shan Li, Hao Zhuang, Thea Aarrestad, Vladimir Loncar, Jennifer Ngadiuba, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers. 2021. Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors. *Nature Mach. Intell.* 3 (2021), 675. doi:10.1038/s42256-021-00356-5 arXiv:2006.10159 [physics.ins-det]
- [30] Claudionor N. Coelho, Aki Kuusela, Shan Li, Hao Zhuang, Jennifer Ngadiuba, Thea Klæboe Aarrestad, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers. 2021. Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors. *Nature Mach. Intell.* 8 (2021), 675. doi:10.1038/s42256-021-00356-5 arXiv:2006.10159 [physics.ins-det]
- [31] ATLAS Collaboration. 2008. The ATLAS Experiment at the CERN Large Hadron Collider. *JINST* 3 (2008), S08003. doi:10.1088/1748-0221/3/08/S08003
- [32] CMS Collaboration. 2008. The CMS experiment at the CERN LHC. *JINST* 3 (2008), S08004. doi:10.1088/1748-0221/3/08/S08004

- [33] Miles Cranmer. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. (2023). arXiv:2305.01582 [astro-ph.IM]
- [34] Allison McCarn Deiana et al. 2022. Applications and Techniques for Fast Machine Learning in Science. *Front. Big Data* 5 (2022), 787421. doi:10.3389/fdata.2022.787421 arXiv:2110.13041 [cs.LG]
- [35] Trysten Dembeck and Chirag Parikh. 2025. Development and Optimization of an Ultra-lightweight Deep Spoken Keyword Spotting Model for FPGA Acceleration. In *Computer Applications in Industry and Engineering*, Gongzhu Hu, Krishna K. Kambhampaty, and Indranil Roy (Eds.). Springer Nature Switzerland, Cham, 3.
- [36] Li Deng. 2012. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141.
- [37] Giuseppe Di Guglielmo et al. 2025. End-to-end workflow for machine learning-based qubit readout with QICK and hls4ml. (1 2025). arXiv:2501.14663 [quant-ph]
- [38] Jennet Dickinson et al. 2023. Smartpixels: Towards on-sensor inference of charged particle track parameters and uncertainties. (12 2023). arXiv:2312.11676 [hep-ex]
- [39] Qader Dorosti. 2025. AI-Enhanced Self-Triggering for Extensive Air Showers: Performance and FPGA Feasibility. (2 2025). arXiv:2502.21198 [astro-ph.IM]
- [40] Javier Duarte et al. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *JINST* 13, 07 (2018), P07027. doi:10.1088/1748-0221/13/07/P07027 arXiv:1804.06913 [physics.ins-det]
- [41] Noemi D'Abbondanza, Stylianos Tzelepis, Nicolò Ghielmetti, Ioannis Kakogeorgiou, Vanya Buchova, Konstantinos Karantzas, Katerina Kikaki, Nicolas-Marcel Lemoine, Maurizio Pierini, Sioni Summers, et al. 2025. Edge SpAlce: Deep Learning Deployment Pipeline for Onboard Data Reduction on Satellite FPGAs. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 1243–1249.
- [42] EdgeCortex Inc. 2025. MERA Compiler and Software Framework. <https://www.edgexcortex.com/en/products/mera>. Accessed April 23, 2025.
- [43] Abdelrahman Elabd et al. 2022. Graph Neural Networks for Charged Particle Tracking on FPGAs. *Front. Big Data* 5 (2022), 828666. doi:10.3389/fdata.2022.828666 arXiv:2112.02048 [physics.ins-det]
- [44] Lyndon Evans and Philip Bryant. 2008. LHC Machine. *JINST* 3, 08 (aug 2008), S08001. doi:10.1088/1748-0221/3/08/S08001
- [45] FastML Team. 2025. *fastmachinelearning/hls4ml*. doi:10.5281/zenodo.1201549
- [46] Fabrizio Ferrandi, Vito Giovanni Castellana, Serena Curzel, Pietro Fezzardi, Michele Fiorito, Marco Lattuada, Marco Minutoli, Christian Pilato, and Antonino Tumeo. 2021. Invited: Bambu: an Open-Source Research Framework for the High-Level Synthesis of Complex Applications. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1327–1330. doi:10.1109/DAC18074.2021.9586110
- [47] Nicolò Ghielmetti et al. 2022. Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml. *Mach. Learn. Sci. Tech.* 3 (2022), 045011. doi:10.1088/2632-2153/ac9cb5 arXiv:2205.07690 [cs.CV]
- [48] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2022. A survey of quantization methods for efficient neural network inference. In *Low-power computer vision*. Chapman and Hall/CRC, Boca Raton, FL, USA, 291–326.
- [49] Fotis I. Giasemis, Vladimir Lončar, Bertrand Granado, and Vladimir Vava Gligorov. 2025. Comparative Analysis of FPGA and GPU Performance for Machine Learning-Based Track Reconstruction at LHCb. (2 2025). arXiv:2502.02304 [hep-ex]
- [50] Davide Giri, Kuan-Lin Chiu, Giuseppe Di Guglielmo, Paolo Mantovani, and Luca P Carloni. 2020. ESP4ML: Platform-based design of systems-on-chip for embedded machine learning. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1049–1054.
- [51] Google. 2025. *Large Language Models (LLMs) with Google AI/ Google Cloud*. Retrieved May 1, 2025 from <https://cloud.google.com/ai/llms>
- [52] Ekaterina Govorkova et al. 2022. Autoencoders on field-programmable gate arrays for real-time, unsupervised new physics detection at 40 MHz at the Large Hadron Collider. *Nature Mach. Intell.* 4 (2022), 154. doi:10.1038/s42256-022-00441-3 arXiv:2108.03986 [physics.ins-det]
- [53] Yijin Guan, Hao Liang, Ningyi Xu, Wenqiang Wang, Shaoshuai Shi, Xi Chen, Guangyu Sun, Wei Zhang, and Jason Cong. 2017. FP-DNN: An Automated Framework for Mapping Deep Neural Networks onto FPGAs with RTL-HLS Hybrid Templates. In *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE Computer Society, Los Alamitos, CA, USA, 152–159. doi:10.1109/FCCM.2017.25
- [54] Yuchen Gui, Qizhe Wu, Wei Yuan, Huawen Liang, Xiaotian Wang, and Xi Jin. 2024. A FPGA-HBM-Based Hardware Streaming Accelerator for GNN Sampling. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, Hong Kong, Hong Kong, 77–78. doi:10.1109/ASAP61560.2024.00026
- [55] Mathew Hall and Vaughn Betz. 2020. HPIPE: Heterogeneous Layer-Pipelined and Sparse-Aware CNN Inference for FPGAs. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Seaside, CA, USA) (FPGA '20). Association for Computing Machinery, New York, NY, USA, 320. doi:10.1145/3373087.3375380
- [56] Yidi Hao, Changqing Feng, Zixuan Zhou, Wenhao Dong, Zhujun Fang, Xueye Hu, Hang Zhou, and Shubin Liu. 2025. A 3-D Track Reconstruction Algorithm for Preresearch of the STCF MDC L1 Trigger. *IEEE Trans. Nucl. Sci.* 72, 3 (2025), 429. doi:10.1109/TNS.2024.

- 3503068
- [57] Ben Hawks, Dennis Plotnikov, Karla Tame-Narvaez, Hamza Ezzaoui Rahali, Mohammad Mehdi Rahimifar, Audrey C. Therrien, Giuseppe Di Guglielmo, Nhan Tran, Javier Duarte, and Vladimir Loncar. 2025. wa-hls4ml and lui-gnn: A benchmark and GNN-based surrogate model for hls4ml resource and latency estimation. Fermi National Accelerator Laboratory (FNAL), Batavia, IL (United States). doi:10.2172/2549315
 - [58] Aram Hayrapetyan et al. 2024. Development of the CMS detector for the CERN LHC Run 3. *JINST* 19, 05 (2024), P05064. doi:10.1088/1748-0221/19/05/P05064 arXiv:2309.05466 [physics.ins-det]
 - [59] Zhenhao He, Dario Korolija, Yu Zhu, Benjamin Ramhorst, Tristan Laan, Lucian Petrica, Michaela Blott, and Gustavo Alonso. 2024. {ACCL+}: an {FPGA-Based} Collective Engine for Distributed Applications. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 211–231.
 - [60] Maximilian Jakob Heer, Benjamin Ramhorst, and Gustavo Alonso. 2025. Machine Learning-based Deep Packet Inspection at Line Rate for RDMA on FPGAs. In *Proceedings of the 5th Workshop on Machine Learning and Systems* (World Trade Center, Rotterdam, Netherlands) (*EuroMLSys '25*). Association for Computing Machinery, New York, NY, USA, 148–155. doi:10.1145/3721146.3721935
 - [61] Ingo Hoyer, Alexander Utz, Christoph Hoog Antink, and Karsten Seidl. 2025. tinyHLS: a novel open source high level synthesis tool targeting hardware accelerators for artificial neural network inference. *Physiological Measurement* 46, 1 (2025), 015002. doi:10.1088/1361-6579/ada8f0
 - [62] Yunxiang Hu, Yuhao Liu, and Zhuoyuan Liu. 2022. A Survey on Convolutional Neural Network Accelerators: GPU, FPGA and ASIC. In *2022 14th International Conference on Computer Research and Development (ICCRD)*. IEEE, Shenzhen, China, 100–107. doi:10.1109/ICCRD54409.2022.9730377
 - [63] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems* 5 (2023), 269–287.
 - [64] Yutaro Iiyama et al. 2020. Distance-Weighted Graph Neural Networks on FPGAs for Real-Time Particle Reconstruction in High Energy Physics. *Front. Big Data* 3 (2020), 598927. doi:10.3389/fdata.2020.598927 arXiv:2008.03601 [physics.ins-det]
 - [65] Intel. 2025. *oneAPI: A New Era of Heterogeneous Computing*. Retrieved May 1, 2025 from <https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html#gs.lz9vkh>
 - [66] Intel Corporation. 2024. Intel FPGA AI Suite. <https://www.intel.com/content/www/us/en/software-kit/851795/intel-fpga-ai-suite.html>. Accessed June 2, 2025.
 - [67] Khayrul Islam, Ryan F. Forelli, Jianzhong Han, Deven Bhadane, Jian Huang, Joshua C. Agar, Nhan Tran, Seda Oğrenci, and Yaling Liu. 2025. Real-Time Cell Sorting with Scalable In Situ FPGA-Accelerated Deep Learning. arXiv:2503.12622 [cs.LG] <https://arxiv.org/abs/2503.12622>
 - [68] Abhishek Kumar Jain, Sharan Kumar, Aashish Tripathi, and Dinesh Gaitonde. 2021. Sparse Deep Neural Network Acceleration on HBM-Enabled FPGA Platform. In *2021 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, Waltham, MA, USA, 1–7. doi:10.1109/HPEC49654.2021.9622804
 - [69] Zhixing Jiang, Dennis Yin, Yihui Chen, Elham E. Khoda, Scott Hauck, Shih-Chieh Hsu, Ekaterina Govorkova, Philip Harris, Vladimir Loncar, and Eric A. Moreno. 2025. Low latency transformer inference on FPGAs for physics applications with hls4ml. *JINST* 20 (2025), P04014. doi:10.1088/1748-0221/20/04/P04014 arXiv:2409.05207 [cs.LG]
 - [70] Zhixing Jiang, Dennis Yin, Elham E. Khoda, Vladimir Loncar, Ekaterina Govorkova, Eric Moreno, Philip Harris, Scott Hauck, and Shih-Chieh Hsu. 2023. Ultra Fast Transformers on FPGAs for Particle Physics Experiments. In *Machine Learning and the Physical Sciences Workshop, NeurIPS*. arXiv:2402.01047 [cs.LG] https://ml4physicsciences.github.io/2023/files/NeurIPS_ML4PS_2023_241.pdf
 - [71] Caroline Johnson. 2023. *Evaluating the Quality of HLS4ML's Basic Neural Network Implementations on FPGAs*. Master's thesis. University of Washington, Dept. of ECE.
 - [72] Caroline Johnson, Scott Hauck, Shih-Chieh Hsu, Waiz Khan, Matthew Bavier, Oleh Kondratyuk, Trinh Nguyen, Stephany Ayala-Cerna, Aidan Short, Jan Silva, Anatoliy Martynyuk, and Geoff Jones. 2023. Quantifying the Efficiency of High-Level Synthesis for Machine Learning Inference. In *Fast ML for Science Workshop at ICCAD*.
 - [73] Matthew Kaufman. 2024. *NASA Trains Machine Learning Algorithm for Mars Sample Analysis*. Retrieved May 6, 2025 from <https://www.nasa.gov/technology/nasa-trains-machine-learning-algorithm-for-mars-sample-analysis/>
 - [74] Elham E. Khoda et al. 2023. Ultra-low latency recurrent neural network inference on FPGAs for physics applications with hls4ml. *Mach. Learn. Sci. Tech.* 4, 2 (2023), 025004. doi:10.1088/2632-2153/acc0d7 arXiv:2207.00559 [cs.LG]
 - [75] Dario Korolija, Timothy Roscoe, and Gustavo Alonso. 2020. Do OS abstractions make sense on FPGAs?. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 56, 20 pages.
 - [76] Lauri Laatu, Chang Sun, Arianna Cox, Abhijith Gandrakota, Benedikt Maier, Jennifer Ngadiuba, Zhiqiang Que, Wayne Luk, Maria Spiropulu, and Alexander Tapper. 2025. Sub-microsecond Transformers for Jet Tagging on FPGAs. arXiv:2510.24784 [physics.ins-det] <https://arxiv.org/abs/2510.24784>
 - [77] Yihang Lu, Xingyun Qi, Yuan Li, Mingche Lai, Yankang Zhao, Zhenqi Li, Hanyuan Li, and Qiang Wang. 2024. Automatic Implementation of Large-Scale CNNs on FPGA Cluster Based on HLS4ML. In *2024 IEEE International Symposium on Parallel and Distributed Processing*

- with Applications (ISPA). IEEE, 1080–1087.
- [78] Paolo Mantovani, Davide Giri, Giuseppe Di Guglielmo, Luca Piccolboni, Joseph Zuckerman, Emilio G Cota, Michele Petracca, Christian Pilato, and Luca P Carloni. 2020. Agile SoC development with open ESP. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.
 - [79] Mohammad Mehdi Rahimifar, Quentin Wingerling, Berthié Gouin-Ferland, Ryan Coffee, and Audrey C. Therrien. 2024. Accelerating data acquisition with FPGA-based edge machine learning: a case study with LCLS-II. *Mach. Learn. Sci. Tech.* 5, 4 (2024), 045041. doi:10.1088/2632-2153/ad8ea8
 - [80] Feroz Ahmed Mian and Saima Zafar. 2024. SoC-Based Implementation of 1-D Convolutional Neural Network for 3-Channel ECG Arrhythmia Classification via HLS4ML. *IEEE Embedded Systems Letters* 16, 4 (2024), 429–432. doi:10.1109/LES.2024.3354081
 - [81] Siqu Miao, Zhiyuan Lu, Mia Liu, Javier Duarte, and Pan Li. 2024. Locality-Sensitive Hashing-Based Efficient Point Transformer with Applications in High-Energy Physics. In *41st International Conference on Machine Learning*, Vol. 235. 35546. arXiv:2402.12535 [cs.LG] <https://proceedings.mlr.press/v235/miao24b.html>
 - [82] Iván René Morales, Romina Soledad Molina, Mladen Bogovac, Nikola Jovalekic, Maria Liz Crespo, Kalliopi Kanaki, Giovanni Ramponi, and Sergio Carrato. 2024. Gamma/Neutron Online Discrimination Based on Machine Learning With CLYC Detectors. *IEEE Trans. Nucl. Sci.* 71, 12 (2024), 2602. doi:10.1109/tns.2024.3498321
 - [83] Anouar Nechi, Lukas Groth, Saleh Mulhem, Farhad Merchant, Rainer Buchty, and Mladen Berekovic. 2023. Fpga-based deep learning inference accelerators: Where are we standing? *ACM Transactions on Reconfigurable Technology and Systems* 16, 4 (2023), 1–32.
 - [84] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning.
 - [85] Jennifer Ngadiuba, Vladimir Loncar, Maurizio Pierini, Sioni Summers, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergio Jindariani, Mia Liu, Kevin Pedro, Nhan Tran, Edward Kreinar, Sheila Sagar, Zhenbin Wu, and Duc Hoang. 2020. Compressing deep neural networks on FPGAs to binary and ternary precision with hls4ml. *Machine Learning: Science and Technology* 2, 1 (dec 2020), 015001. doi:10.1088/2632-2153/aba042
 - [86] Patrick Odagiu et al. 2024. Ultrafast jet classification at the HL-LHC. *Mach. Learn. Sci. Tech.* 5 (2024), 035017. doi:10.1088/2632-2153/ad5f10 arXiv:2402.01876 [hep-ex]
 - [87] Alessandro Pappalardo. 2023. *Xilinx/brevitas*. doi:10.5281/zenodo.3333552
 - [88] Alessandro Pappalardo, Yaman Umuroglu, Michaela Blott, Jovan Mitrevski, Ben Hawks, Nhan Tran, Vladimir Loncar, Sioni Summers, Hendrik Borras, Jules Muhizi, Matthew Trahms, Shih-Chieh Hsu Hsu, Scott Hauck, and Javier Duarte. 2022. QONNX: Representing Arbitrary-Precision Quantized Neural Networks. In *4th Workshop on Accelerated Machine Learning (AccML) at HiPEAC 2022 Conference*. arXiv:2206.07527 [cs.LG] [https://accml.dcs.gla.ac.uk/papers/2022/4thAccML_paper_1\(12\).pdf](https://accml.dcs.gla.ac.uk/papers/2022/4thAccML_paper_1(12).pdf)
 - [89] Benjamin Parpillon, Chinar Syal, Jieun Yoo, Jennet Dickinson, Morris Swartz, Giuseppe Di Guglielmo, Alice Bean, Douglas Berry, Manuel Blanco Valentin, Karri DiPetrillo, Anthony Badea, Lindsey Gray, Petar Maksimovic, Corrinne Mills, Mark S. Neubauer, Gauri Pradhan, Nhan Tran, Dahai Wen, and Farah Fahim. 2024. Smart Pixels: In-pixel AI for on-sensor data filtering. arXiv:2406.14860 [physics.ins-det] <https://arxiv.org/abs/2406.14860>
 - [90] Maurizio Pierini, Javier Mauricio Duarte, Nhan Tran, and Marat Freytsis. 2020. HLS4ML LHC Jet dataset (150 particles). doi:10.5281/zenodo.3602260
 - [91] Patrick Plagwitz, Frank Hannig, Martin Ströbel, Christoph Strohmeier, and Jürgen Teich. 2021. A Safari through FPGA-based Neural Network Compilation and Design Automation Flows. In *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 10–19. doi:10.1109/FCCM51124.2021.00010
 - [92] Zhiqiang Que, Jose GF Coutinho, Ce Guo, Hongxiang Fan, and Wayne Luk. [n. d.]. MetaML-Pro: Cross-Stage Design Flow Automation for Efficient Deep Learning Acceleration. *ACM Transactions on Reconfigurable Technology and Systems* ([n. d.]).
 - [93] Zhiqiang Que, Hongxiang Fan, Gabriel Figueiredo, Ce Guo, Wayne Luk, Ryota Yasudo, and Masato Motomura. 2025. Trustworthy Deep Learning Acceleration with Customizable Design Flow Automation. In *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*. 1–13.
 - [94] Zhiqiang Que, Shuo Liu, Markus Rognlien, Ce Guo, Jose GF Coutinho, and Wayne Luk. 2023. Metaml: Automating customizable cross-stage design-flow for deep learning acceleration. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 248–252.
 - [95] Zhiqiang Que, Chang Sun, Sudarshan Paramesvaran, Emyr Clement, Katerina Karakoulaki, Christopher Brown, Lauri Laatu, Arianna Cox, Alexander Tapper, Wayne Luk, and Maria Spiropulu. 2025. JEDI-linear: Fast and Efficient Graph Neural Networks for Jet Tagging on FPGAs. In *2025 International Conference on Field-Programmable Technology (FPT)*. IEEE.
 - [96] Mohammad Mehdi Rahimifar, Hamza Ezzaoui Rahali, and Audrey C. Therrien. 2024. rule4ml: An Open-Source Tool for Resource Utilization and Latency Estimation for ML Models on FPGA. arXiv:2408.05314 [cs.LG] <https://arxiv.org/abs/2408.05314>
 - [97] Benjamin Ramhorst, Dario Korolija, Maximilian Jakob Heer, Jonas Dann, Luhao Liu, and Gustavo Alonso. 2025. Coyote v2: Raising the Level of Abstraction for Data Center FPGAs (SOSP '25). Association for Computing Machinery, New York, NY, USA, 639–654. doi:10.1145/3731569.3764845

- [98] Benjamin Ramhorst, Vladimir Lončar, and George A. Constantinides. 2023. FPGA Resource-aware Structured Pruning for Real-Time Neural Networks. In *2023 International Conference on Field Programmable Technology (ICFPT)*. IEEE, Yokohama, Japan, 282. doi:10.1109/icfpt59805.2023.00046
- [99] Mohammad Riazati, Masoud Daneshthalab, Mikael Sjödin, and Björn Lisper. 2020. DeepHLS: A complete toolchain for automatic synthesis of deep neural networks to FPGA. In *2020 27th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. IEEE, Glasgow, UK, 1–4. doi:10.1109/ICECS49266.2020.9294881
- [100] Amazon Web Services. 2021. *Seafloor Systems Saves 4 Hours of Labor per Robot Build Using AWS IoT Greengrass*. Retrieved May 6, 2025 from <https://aws.amazon.com/solutions/case-studies/seafloor/>
- [101] Amazon Web Services. 2024. *Siemens Electronics Factory Erlangen Reduces Machine Learning Deployment Time by 80% with AWS and Siemens Industrial AI on Industrial Edge*. Retrieved May 6, 2025 from <https://aws.amazon.com/partners/success/siemens-electronics-factory-erlangen-siemens/>
- [102] Amazon Web Services. 2025. *Transform your business with generative AI*. Retrieved May 1, 2025 from <https://aws.amazon.com/ai/generative-ai/>
- [103] Hardik Sharma, Jongse Park, Divya Mahajan, Emmanuel Amaro, Joon Kyung Kim, Chenkai Shao, Asit Mishra, and Hadi Esmaeilzadeh. 2016. From high-level deep neural models to FPGAs. In *The 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)*. IEEE Press, Taipei, Taiwan, Article 17, 12 pages.
- [104] Yilin Shen. 2024. *Evaluating the Efficiency of Neural Network Implementations on AMD Versal AI Engines*. Master's thesis. University of Washington, Dept. of ECE.
- [105] R. Shi, S. Ogrenci, J. M. Arnold, J. R. Berlioz, P. Hanlet, K. J. Hazelwood, M. A. Ibrahim, H. Liu, V. P. Nagaslaev, A. Narayanan 1, D. J. Nicklaus, J. Mitrevski, G. Pradhan, A. L. Saewert, B. A. Schupbach, K. Seiya, M. Thieme, R. M. Thurman-Keup, and N. V. Tran. 2023. ML-based Real-Time Control at the Edge: An Approach Using hls4ml. arXiv:2311.05716 [cs.AR] <https://arxiv.org/abs/2311.05716>
- [106] Siemens Software. 2025. Catapult High-Level Synthesis and Verification. <https://eda.sw.siemens.com/en-US/ic/catapult-high-level-synthesis/>. Accessed May 30, 2025.
- [107] Jon Slominski and Brad Bonn. 2021. *How Boston Dynamics and AWS use mobility and computer vision for dynamic sensing*. Retrieved May 6, 2025 from <https://aws.amazon.com/blogs/robotics/how-boston-dynamics-and-aws-use-mobility-and-computer-vision-for-dynamic-sensing/>
- [108] Edge SpAlce. 2024. *Novel Edge-AI system for accurate and near real-time plastic detection and monitoring in marine environment*. Retrieved May 1, 2025 from <https://edgespaice.eu/>
- [109] Jason St. John et al. 2021. Real-time artificial intelligence for accelerator control: A study at the Fermilab Booster. *Phys. Rev. Accel. Beams* 24 (2021), 104601. doi:10.1103/PhysRevAccelBeams.24.104601 arXiv:2011.07371 [physics.acc-ph]
- [110] Marius Stan, Mathew Hall, Mohamed Ibrahim, and Vaughn Betz. 2022. HPIPE NX: Boosting CNN Inference Acceleration Performance with AI-Optimized FPGAs. In *2022 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, Hong Kong, Hong Kong, 1–9. doi:10.1109/ICFPT56656.2022.9974441
- [111] Chang Sun, Takumi Nakajima, Yuki Mitsumori, Yasuyuki Horii, and Makoto Tomoto. 2023. Fast muon tracking with machine learning implemented in FPGA. *Nucl. Instrum. Meth. A* 1045 (2023), 167546. doi:10.1016/j.nima.2022.167546 arXiv:2202.04976 [physics.ins-det]
- [112] Chang Sun, Jennifer Ngadiuba, Maurizio Pierini, and Maria Spiropulu. 2025. Fast jet tagging with MLP-Mixers on FPGAs. *Machine Learning: Science and Technology* 6, 3 (2025), 035025.
- [113] Chang Sun, Zhiqiang Que, Thea Aarrestad, Vladimir Loncar, Jennifer Ngadiuba, Wayne Luk, and Maria Spiropulu. 2026. HGQ: High Granularity Quantization for Real-time Neural Networks on FPGAs. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 79–91.
- [114] Chang Sun, Zhiqiang Que, Vladimir Loncar, Wayne Luk, and Maria Spiropulu. 2025. da4ml: Distributed arithmetic for real-time neural networks on fpgas. *ACM Transactions on Reconfigurable Technology and Systems* (2025).
- [115] Shinya Takamaeda-Yamazaki. 2017. NNgen: A Fully-Customizable Hardware Synthesis Compiler for Deep Neural Network. <https://github.com/NNgen/nnngen>. Accessed April 23, 2025.
- [116] Naif Tarafdar, Giuseppe Di Guglielmo, Philip C Harris, Jeffrey D Krupa, Vladimir Loncar, Dylan S Rankin, Nhan Tran, Zhenbin Wu, Qianfeng Shen, and Paul Chow. 2021. Aigean: An open framework for deploying machine learning on heterogeneous clusters. *ACM Transactions on Reconfigurable Technology and Systems (TRETs)* 15, 3 (2021), 1–32.
- [117] Riccardo Testa, Mohamad Yaacoub, Christian Gianoglio, and Maurizio Valle. 2025. *FPGA Implementation of a Convolutional Recurrent Neural Network for Real-Time Sensor Data Processing*. Springer, Cham, 258. doi:10.1007/978-3-031-71518-1_30
- [118] Neil C. Thompson and Svenja Spanuth. 2021. The decline of computers as a general purpose technology. *Commun. ACM* 64, 3 (Feb. 2021), 64–72.
- [119] Ho Fung Tsoi, Vladimir Loncar, Sridhara Dasu, and Philip Harris. 2025. SymbolNet: neural symbolic regression with adaptive dynamic pruning for compression. *Mach. Learn. Sci. Tech.* 6 (2025), 015021. doi:10.1088/2632-2153/adaad8 arXiv:2401.09949 [cs.LG]
- [120] Ho Fung Tsoi, Adrian Alan Pol, Vladimir Loncar, Ekaterina Govorkova, Miles Cranmer, Sridhara Dasu, Peter Elmer, Philip Harris, Isobel Ojalvo, and Maurizio Pierini. 2024. Symbolic Regression on FPGAs for Fast Machine Learning Inference. *EPJ Web Conf.* 295

- (2024), 09036. doi:10.1051/epjconf/202429509036 arXiv:2305.04099 [cs.LG]
- [121] Yaman Umuroglu, Yash Akhauri, Nicholas James Fraser, and Michaela Blott. 2020. LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, IEEE, Gothenburg, Sweden, 291–297. doi:10.1109/FPL50879.2020.00055
 - [122] Yaman Umuroglu, Hendrik Borras, Vladimir Loncar, Sioni Summers, and Javier Duarte. 2022. *fastmachinelearning/qonnx*. doi:10.5281/zenodo.7622236
 - [123] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)*. ACM, New York, NY, USA, 65.
 - [124] Yasuhiko NAKASHIMA Van-Cam NGUYEN. 2023. Implementation of Fully-Pipelined CNN Inference Accelerator on FPGA and HBM2 Platform. *IEICE TRANSACTIONS on Information* E106-D, 6 (June 2023), 1117–1129. doi:10.1587/transinf.2022EDP7155
 - [125] Stylianos I. Venieris and Christos-Savvas Bouganis. 2016. fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs. In *Proceedings of the 2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Washington, DC, USA, 40–47. doi:10.1109/FCCM.2016.22
 - [126] Stylianos I. Venieris and Christos-Savvas Bouganis. 2016. fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Washington, DC, USA, 40–47. doi:10.1109/FCCM.2016.22
 - [127] Stylianos I. Venieris and Christos-Savvas Bouganis. 2017. Latency-Driven Design for FPGA-based Convolutional Neural Networks. In *Proceedings of the 2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, Ghent, Belgium, 1–8. doi:10.23919/FPL.2017.8056828
 - [128] Stylianos I. Venieris and Christos-Savvas Bouganis. 2019. fpgaConvNet: Mapping Regular and Irregular Convolutional Neural Networks on FPGAs. *IEEE Transactions on Neural Networks and Learning Systems* 30, 2 (2019), 326–342. doi:10.1109/TNNLS.2018.2856369
 - [129] Jure Vreča and Anton Biasizzo. 2023. Towards Deploying Highly Quantized Neural Networks on FPGA Using Chisel. In *2023 26th Euromicro Conference on Digital System Design (DSD)*. IEEE, Golem, Albania, 161–167. doi:10.1109/DSD60849.2023.00032
 - [130] Bernardita Stitić, Luca Urbinati, Giuseppe Di Guglielmo, Luca P. Carloni, and Mario R. Casu. 2024. Enhanced Machine-Learning Flow for Microwave-Sensing Systems for Contaminant Detection in Food. *IEEE Trans. AgriFood Electron.* 2, 2 (2024), 181. doi:10.1109/TAFE.2024.3421238
 - [131] Dong Wang, Ke Xu, and Diankun Jiang. 2017. PipeCNN: An OpenCL-based open-source FPGA accelerator for convolution neural networks. In *2017 International Conference on Field Programmable Technology (ICFPT)*. IEEE, Melbourne, VIC, Australia, 279–282. doi:10.1109/FPT.2017.8280160
 - [132] Erwei Wang, James J. Davis, Daniele Moro, Piotr Zielinski, Jia Jie Lim, Claudionor Coelho, Satrajit Chatterjee, Peter Y. K. Cheung, and George A. Constantinides. 2023. Enabling Binary Neural Network Training on the Edge. *ACM Trans. Embed. Comput. Syst.* 22, 6, Article 105 (Nov. 2023), 19 pages. doi:10.1145/3626100
 - [133] Ying Wang, Jie Xu, Yinhe Han, Huawei Li, and Xiaowei Li. 2016. DeepBurning: automatic generation of FPGA-based learning accelerators for the neural network family. In *Proceedings of the 53rd Annual Design Automation Conference (Austin, Texas) (DAC '16)*. Association for Computing Machinery, New York, NY, USA, Article 110, 6 pages. doi:10.1145/2897937.2898003
 - [134] Y. Wei, R. F. Forelli, C. Hansen, J. P. Levesque, N. Tran, J. C. Agar, G. Di Guglielmo, M. E. Mauel, and G. A. Navratil. 2024. Low latency optical-based mode tracking with machine learning deployed on FPGAs on a tokamak. *Review of Scientific Instruments* 95, 7 (07 2024), 073509. doi:10.1063/5.0190354 arXiv:2312.00128
 - [135] Jason Weitz, Dmitri Demler, Luke McDermott, Nhan Tran, and Javier Duarte. 2025. Neural Architecture Codesign for Fast Physics Applications. (2025). arXiv:2501.05515 [cs.LG]
 - [136] Jieun Yoo, Jennet Dickinson, Morris Swartz, Giuseppe Di Guglielmo, Alice Bean, Douglas Berry, Manuel Blanco Valentin, Karri DiPetrillo, Farah Fahim, Lindsey Gray, et al. 2024. Smart pixel sensors: towards on-sensor filtering of pixel clusters with deep learning. *Mach. Learn. Sci. Tech.* 5 (2024). doi:10.1088/2632-2153/ad6a00 arXiv:2310.02474 [physics.ins-det]
 - [137] Cheng Zhang, Jianyi Cheng, Zhewen Yu, and Yiren Zhao. 2023. MASE: An Efficient Representation for Software-Defined ML Hardware System Exploration. In *Machine Learning for Systems 2023*. <https://openreview.net/forum?id=Z7v6mxNVdU>
 - [138] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81. doi:10.1016/j.aiopen.2021.01.001